



CS 329Z:Engineering AI Agents

RAG + Agents

Diyi Yang

Announcements

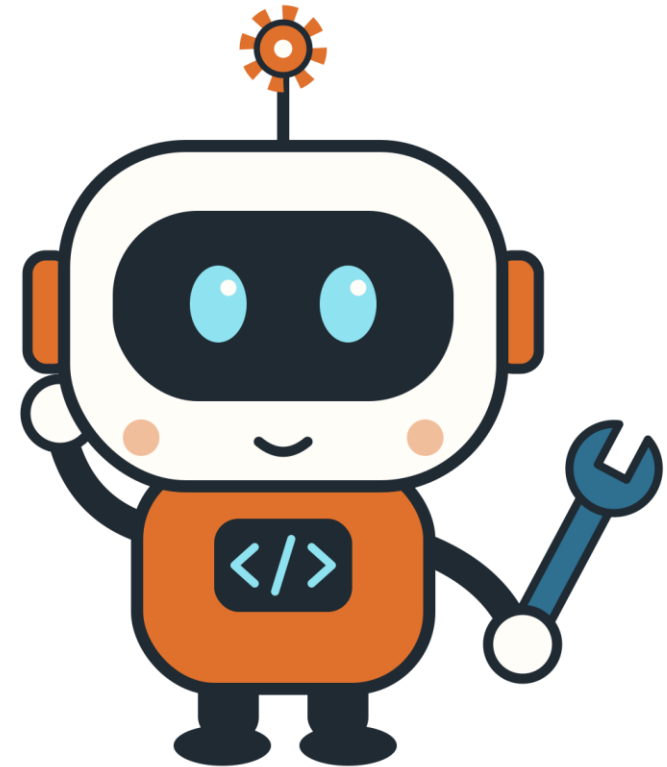
- Waitlist and Audit
- Compute (see Canvas/email)
- Office hours (hybrid: half Zoom; half in-person)
- Homework 1 will be released on Oct 5
- Project proposal due on Oct 9

Lecture Overview

- Project (5 mins)
- Why RAG (5 mins)
- The RAG pipeline (10 mins)
- Indexing: chunking, contextual retrieval (15 mins)
- Retrieval: BM25, DPR, ColBERT (20 mins)
- Agentic RAG: Self-RAG, Search-R1 (10 mins)
- Failure modes (15 mins)

Pick A Topic that You're Excited About

- What task does your agent do?
- What research question drives it?
- What data or APIs does it need?
- Which frameworks will you use?
- How will you evaluate it?



Pick A Topic that You're Excited About

- **Goal:** design, build, and evaluate an agentic system of your own
- **Focus:** building agents (retrieval, tool use, memory, multi-agent systems, optimization), the data agents need, or evaluating and making agents safe.
- **Rigorous evaluation:**
 - A demo that works once will not score highly
 - **Define early:** tasks, where the data comes from, what counts as success
 - **Measure:** code graders, LLM-as-judge, human ratings, or a mix
 - **Report:** many tasks, baselines, and where and why the agent fails

Example Topics for CS329Z Course Projects

Research topics

-  Evaluating agents on cost, latency, and safety
-  Adapting to new tools, APIs, or UIs
-  Robustness to prompt injection and tool errors
-  Why agents misuse tools
-  Learning tool use from little data
-  Interpreting agent behaviors
-  Long-tail and long-horizon edge cases
-  Human-agent/agent-agent teaming
-  Multi-agent systems, memory, planning, oversight, reward hacking

Example ideas

-  Syllabus reader that puts deadlines on your calendar
-  Course-schedule optimizer for next quarter
-  Research-paper discovery and summarization agent
-  Campus-event aggregator and recommender
-  Coding agent specialized for one codebase, framework, or language
-  Rigorous evaluation of an existing agent that surfaces its failure modes

Project Team and Key Considerations

- Recommend 1-3 people
- Clearly divide the work among team members
- Reach out to us if you need anything
- Feel free to talk to your neighbors or use Ed Discussion
- Consider the availability of data, agentic frameworks, computation, and evaluation
- Example project ideas are coming later this week

Recommendations for Successful Projects

- Start early and work on it every week rather than rushing at the end
- Get your data first!
- Have a clear, well-defined research question (creative ones ++)
- **Results should teach us something**
- Present and visualize results well
- Come to office hours and talk to us!

Lecture Overview

- Project (5 mins)
- Why RAG (5 mins)
- The RAG pipeline (10 mins)
- Indexing: chunking, contextual retrieval (15 mins)
- Retrieval: BM25, DPR, ColBERT (20 mins)
- Agentic RAG: Self-RAG, Search-R1 (10 mins)
- Failure modes (15 mins)

Learning Objective: learn how the RAG pipeline is built and where it breaks, what to index, and how to search it

Why Retrieval?

“Which office hour this week is least crowded?”

LLMs Alone

“Try the Wednesday afternoon session – midweek hours are usually quietest.”

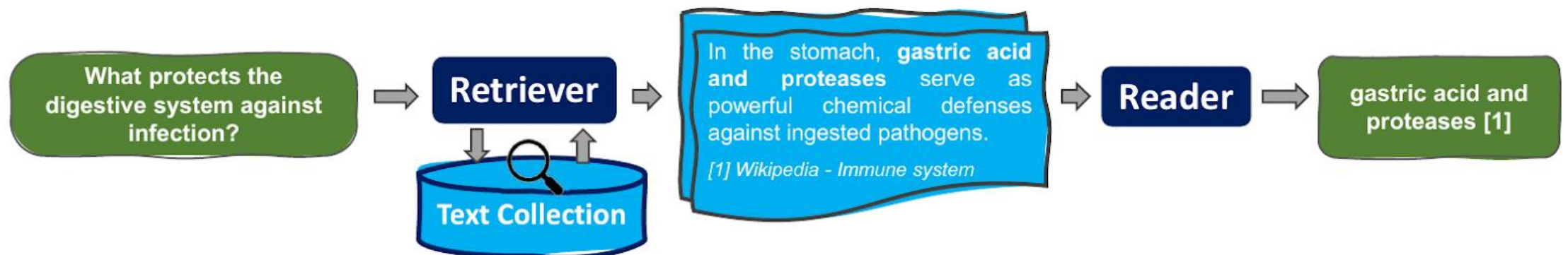
LLMs + Retrieval

Retrieved: “Queue log, last 3 weeks — Tue 4pm: 14 waiting avg; Thu 9am: 3.”

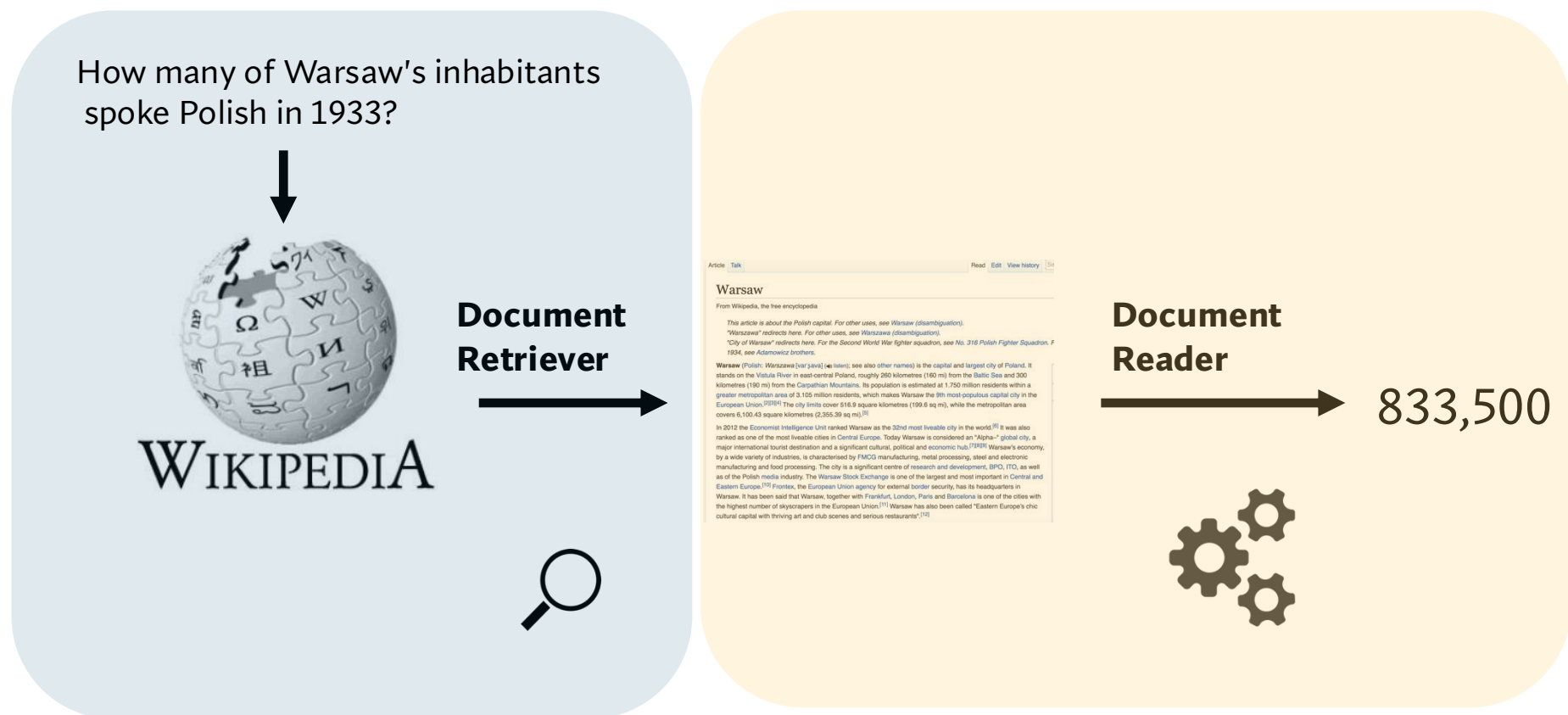
“Thursday 9am — about 3 students ahead of you.”

Using Retrieval to Overcome LMs' Shortcomings

- Instead of asking the LM to memorize everything, can we provide the LM with relevant and useful content just-in-time?
- Retrieval is a common mechanism for identifying such relevant information.
 - **Dynamic:** it's easy to update / add documents to your retrieval system
 - **Interpretable:** LM can generate pointers to retrieved documents that support human verification of its generations (citations)



Retrieval Often Used in Open-Domain QA



<https://github.com/facebookresearch/DrQA>

Chen et al., 2017. Reading Wikipedia to Answer Open-domain Questions

Retriever-Reader Framework

Input: a large collection of documents $\mathcal{D} = D_1, D_2, \dots, D_N$ and Q

Output: an answer string A

Retriever: $f(\mathcal{D}, Q) \rightarrow P_1, \dots, P_K$ K is pre-defined (e.g., 100)

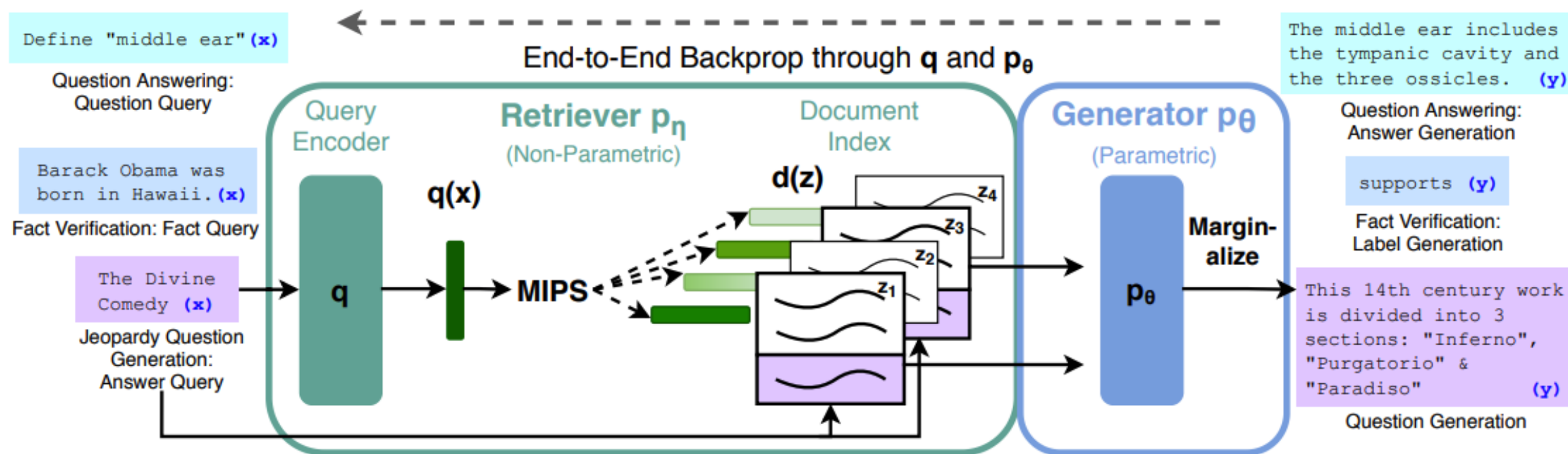
Reader: $g(Q, \{P_1, \dots, P_K\}) \rightarrow A$ A reading comprehension problem!

Retriever = A standard TF-IDF information-retrieval sparse model (a fixed module)

Reader = a neural reading comprehension model

- Could be trained on SQuAD and other distantly-supervised QA datasets
- Or a zero-shot LLM (ChatGPT etc)

Retrieval Augmented Generation (RAG)



See also works like REALM, DPR, etc..

[[Lewis et al., 2021](#)]

Lecture Overview

- Why RAG (5 mins)
- The RAG pipeline (10 mins)
- Indexing: chunking, contextual retrieval (15 mins)
- Retrieval: BM25, DPR, ColBERT (20 mins)
- Agentic RAG: Self-RAG, Search-R1 (10 mins)
- Failure modes (15 mins)

Deep Dive into RAG

- A **retriever** $p(z|x)$ that returns the top-K passages for a query x
- A **generator** $p(y|x, z)$ that writes the answer one token at a time, given the query x and a retrieved passage z

$$p(y|x) \approx \sum_{z \in \text{top-K}} p(z|x)p(y|x, z)$$

Weight each passage's answer by how likely the retriever rates that passage

RAG-Sequence: pick one passage, generate the whole answer with it.

RAG-Token: re-pick the passage at every token of the answer

Deep Dive into RAG

- A **retriever** $p(z|x)$ that returns the top-K passages for a query x
- A **generator** $p(y|x, z)$ that writes the answer one token at a time, given the query x and a retrieved passage z
- Jointly train the retriever and generator to minimize the negative marginal log-likelihood $\sum_j -\log p(y_j|x_j)$

Retriever: DPR

- A retriever $p(z|x)$ that returns the top-K passages for a query x

$$p(z|x) \propto \exp(\mathbf{d}(z)^\top \mathbf{q}(x)) \quad \mathbf{d}(z) = \text{BERT}_d(z), \quad \mathbf{q}(x) = \text{BERT}_q(x)$$

- $\text{top-}k(p(\cdot | x))$: the k highest-scoring passages, solved using Maximum Inner Product Search (MIPS)

Generator: BART

- A generator $p(y|x, z)$ that writes the answer one token at a time, given the query x and a retrieved passage z
- The generator can be modeled using any encoder-decoder
- x and z are concatenated when generating from BART

RAG Performance on Open-Domain QA

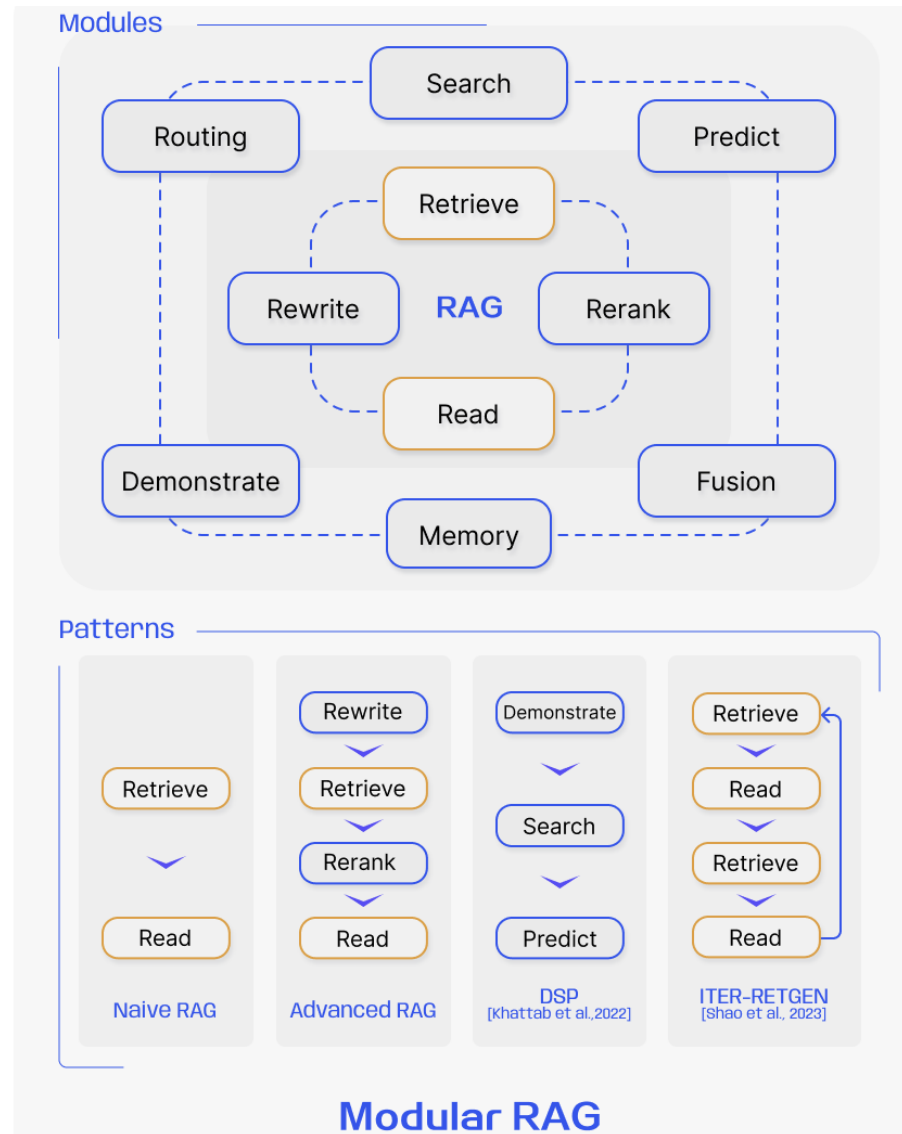
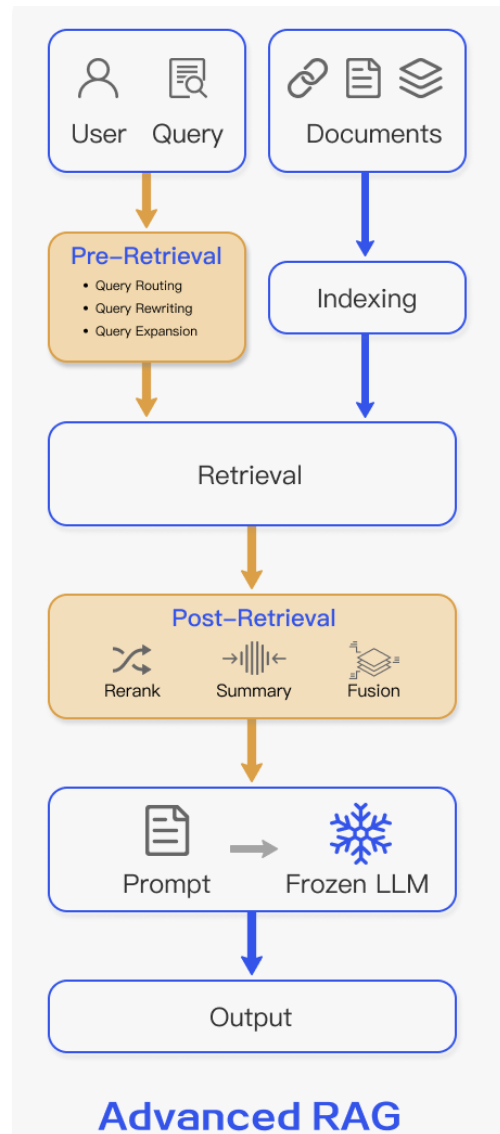
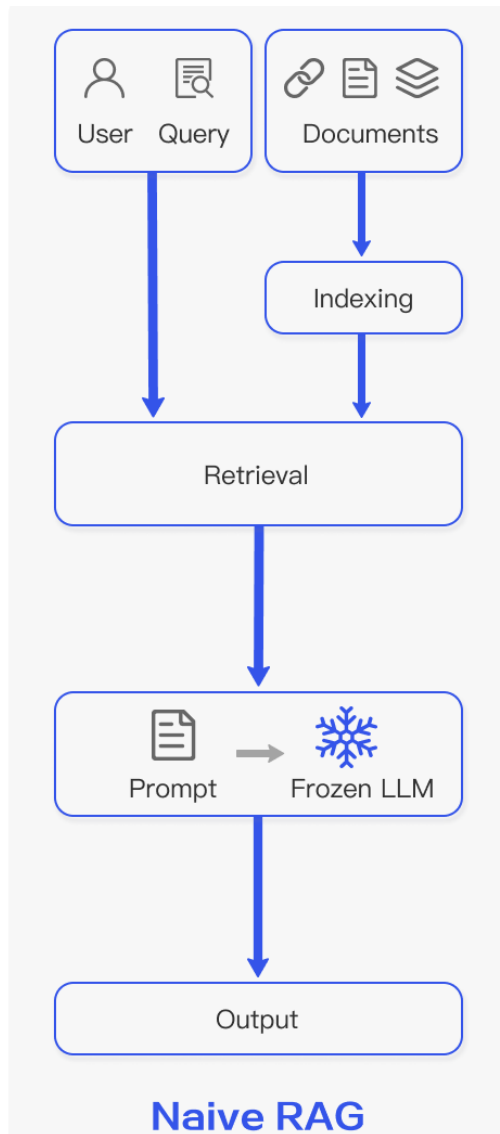
	Model	NQ	TQA	WQ	CT
Closed Book	T5-11B [52]	34.5	- /50.1	37.4	-
	T5-11B+SSM[52]	36.6	- /60.5	44.7	-
Open Book	REALM [20]	40.4	- / -	40.7	46.8
	DPR [26]	41.5	57.9 / -	41.1	50.6
	RAG-Token	44.1	55.2/66.1	45.5	50.0
	RAG-Seq.	44.5	56.8/ 68.0	45.2	52.2

	Factuality	Specificity
BART better	7.1%	16.8%
RAG better	42.7%	37.4%
Both good	11.7%	11.8%
Both poor	17.7%	6.9%
No majority	20.8%	20.1%

RAG on Natural Questions (NQ), TriviaQA (TQA), WebQuestions (WQ) and CuratedTrec (CT), using Exact Match score

Human assessments for the Jeopardy question generation task

Different Variants of RAG (Gao et al, 2024)



Lecture Overview

- Why RAG (5 mins)
- The RAG pipeline (10 mins)
- Indexing: chunking, contextual retrieval (15 mins)
- Retrieval: BM25, DPR, ColBERT (20 mins)
- Agentic RAG: Self-RAG, Search-R1 (10 mins)
- Failure modes (15 mins)

What Makes A Good Retriever? Embeddings

Type	Examples	One vector covers
Static word	word2vec, GloVe	a word type
Contextual	BERT, RoBERTa	a token in context
Passage	Sentence-BERT, DPR, E5	a chunk
Multimodal	CLIP	text or image, shared space
Graph	node2vec, GNN encoders	a node or relation

Retrieval needs both a fixed-dimension vector per chunk, and a similarity metric (cosine or inner product).

Chunking: Basic Unit in Retrieval

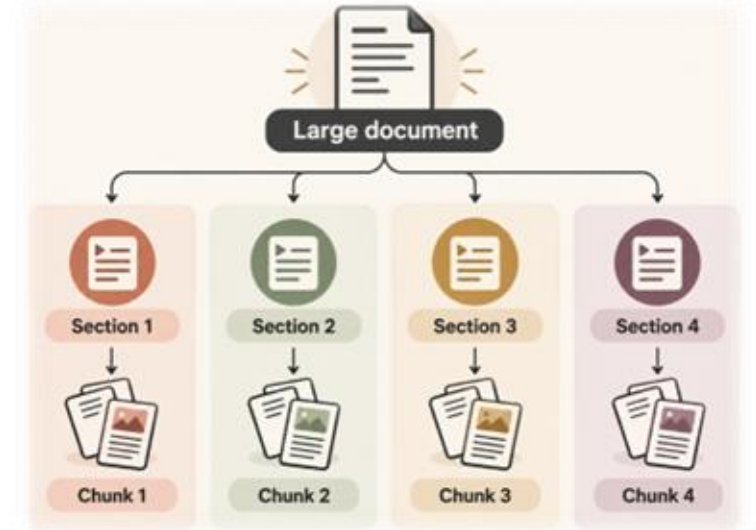
Fixed-size chunking:

- Segment based on size (e.g., 100–500 tokens)
 - *Too small*: the vector has no context
 - *Too large*: one vector averages many topics
- *Overlap*: 10–20%; no sentence lost at chunk boundary

Semantic chunking: segment via logical boundaries (e.g., sentences, sections)

Context-enriched chunking: carry additional metadata or summaries per chunk

AI-driven dynamic chunking: use LLMs to detect natural breakpoints



Chunking: Same Text, Three Splits

Grading. The project milestone is due Week 6, Thursday 11:59pm. Late days may not be applied to the milestone. The final report is due Week 10.

Fixed-size chunking (50 tokens)

... The project milestone is
due Week 6, Thursday

11:59pm. Late days may not be
applied to the milestone ...

Fixed-size chunking + overlap

... The project milestone is
due Week 6, Thursday 11:59pm.

... due Week 6, Thursday
11:59pm. Late days may not be
applied ...

Structure (split on headings)

Grading – Milestone: Week 6,
Thu 11:59pm. Late days: not
allowed on the milestone.
Final report: Week 10.

Why Contextual Retrieval

Query: “What was the revenue growth for ACME Corp in Q2 2023?”

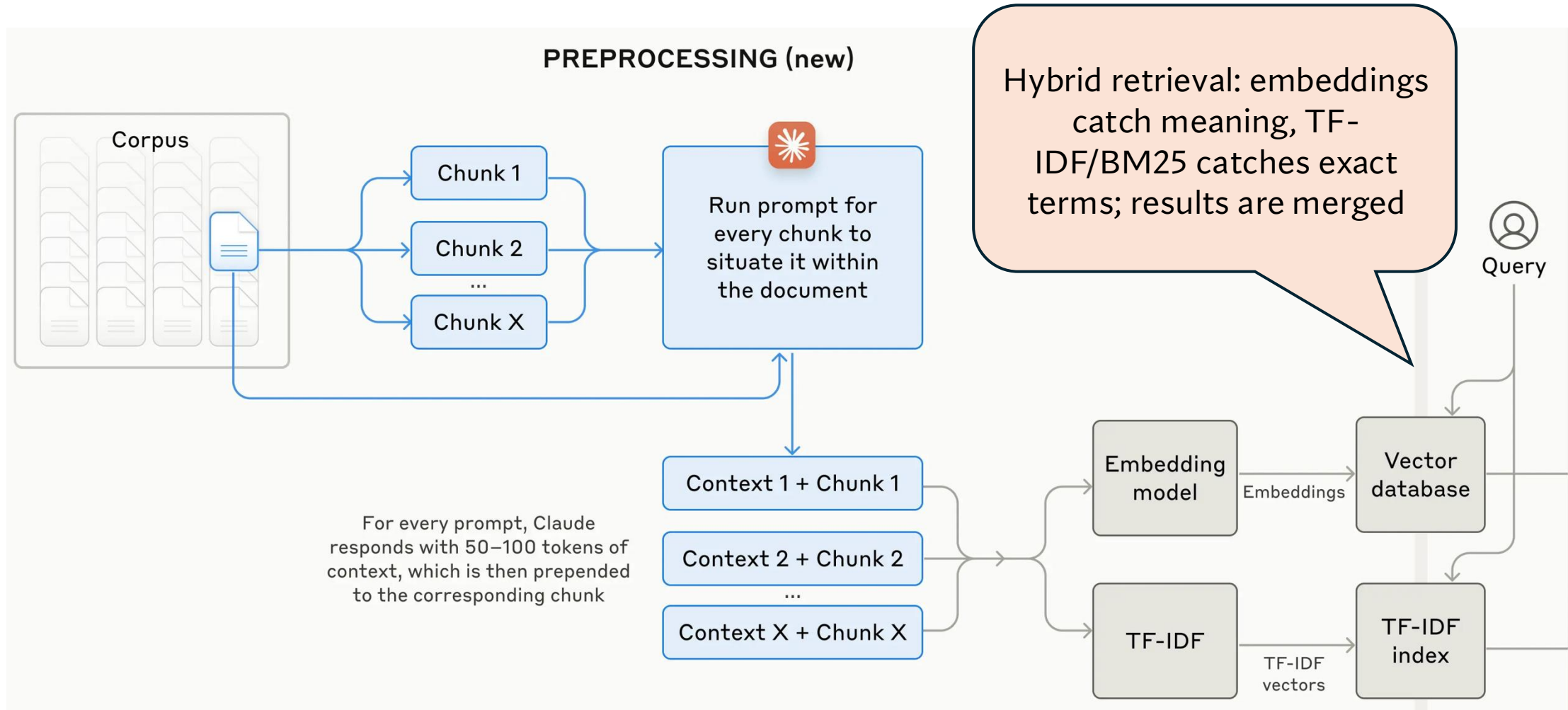
Chunk

“The company’s revenue grew by 3% over the previous quarter.”

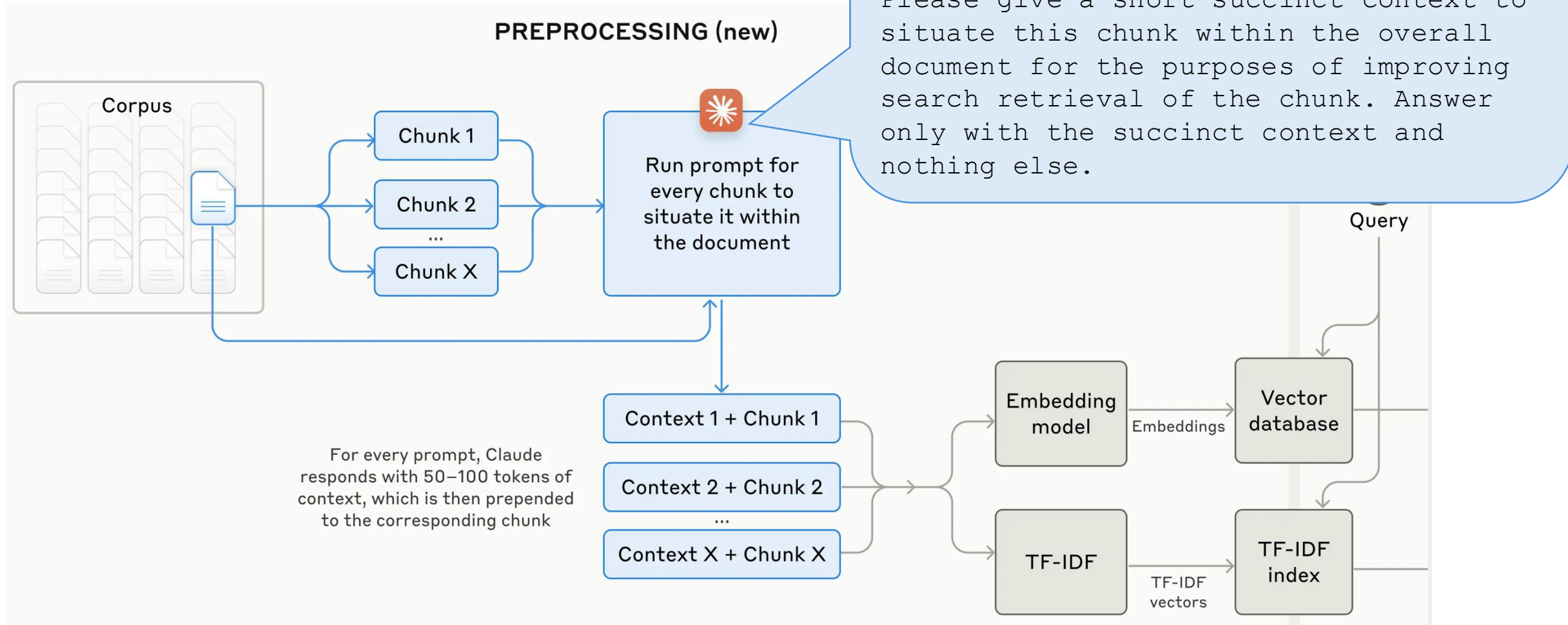
Contextualized Chunk

“From ACME Corp’s Q2 2023 SEC filing; Q1 2023 revenue was \$314M. The company’s revenue grew by 3% over the previous quarter.”

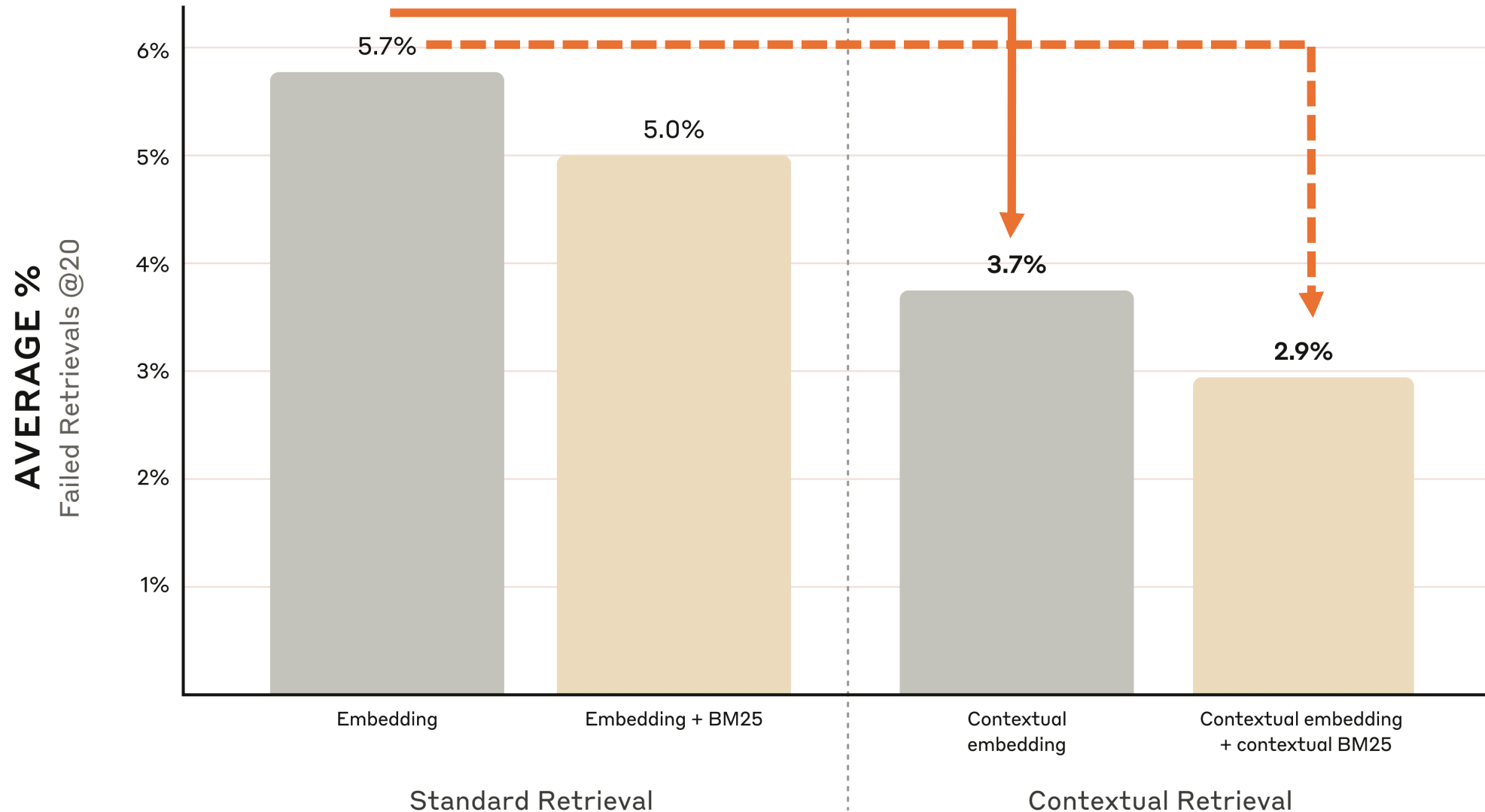
Contextual Retrieval



Contextual Retrieval



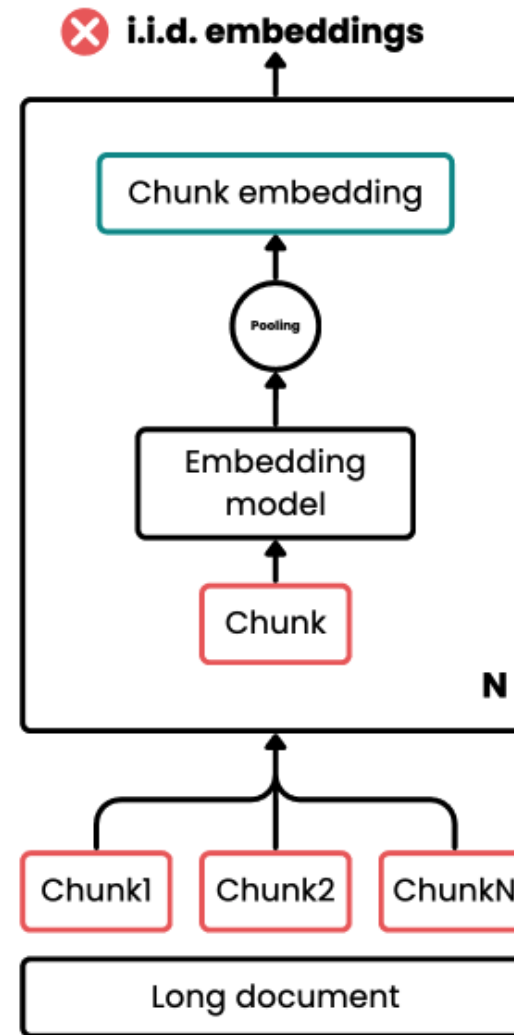
Contextual Embeddings Reduced Retrieval Failure Rate



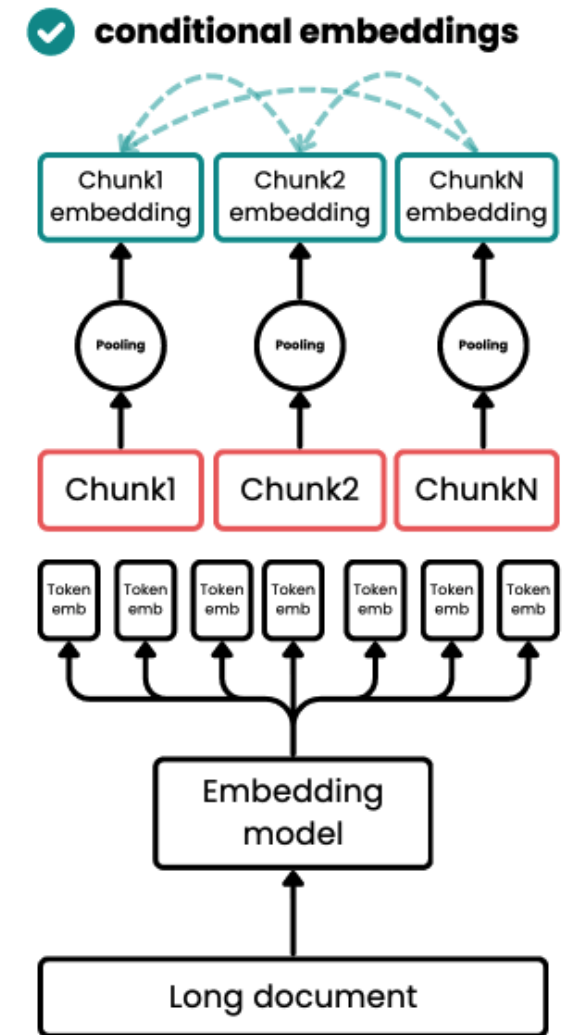
Late Chunking

- **Late chunking:** embed the whole document with a long-context encoder; then mean-pool the token vectors within each chunk's span
- Compared to *Contextual Retrieval*: no LLM call and no added tokens per chunk; the context is carried in the vector

(Günther et al., 2024)



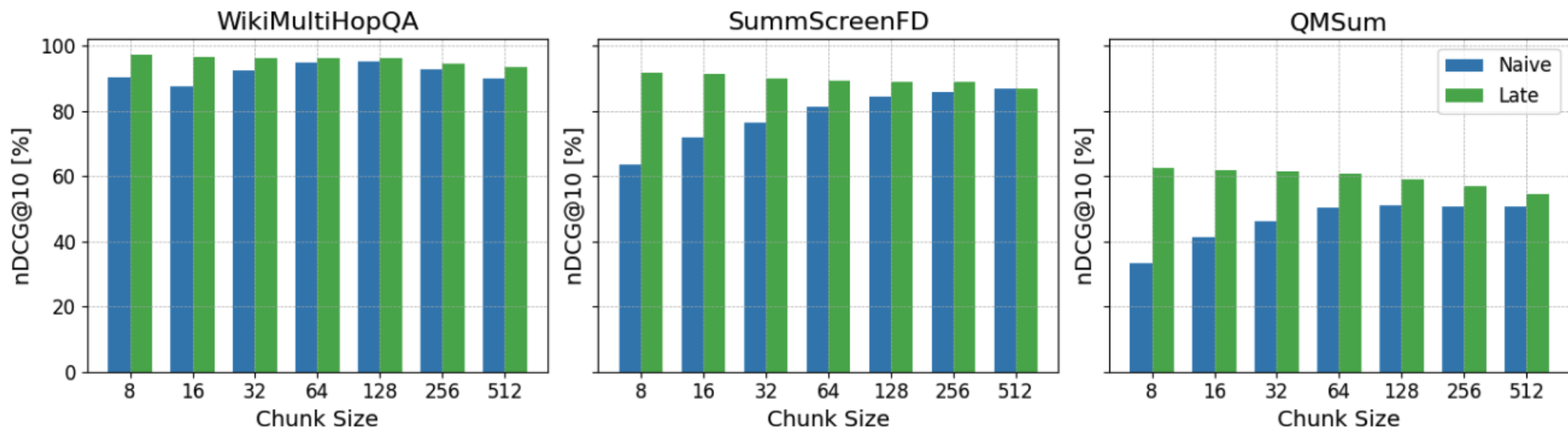
NAIVE CHUNKING



LATE CHUNKING

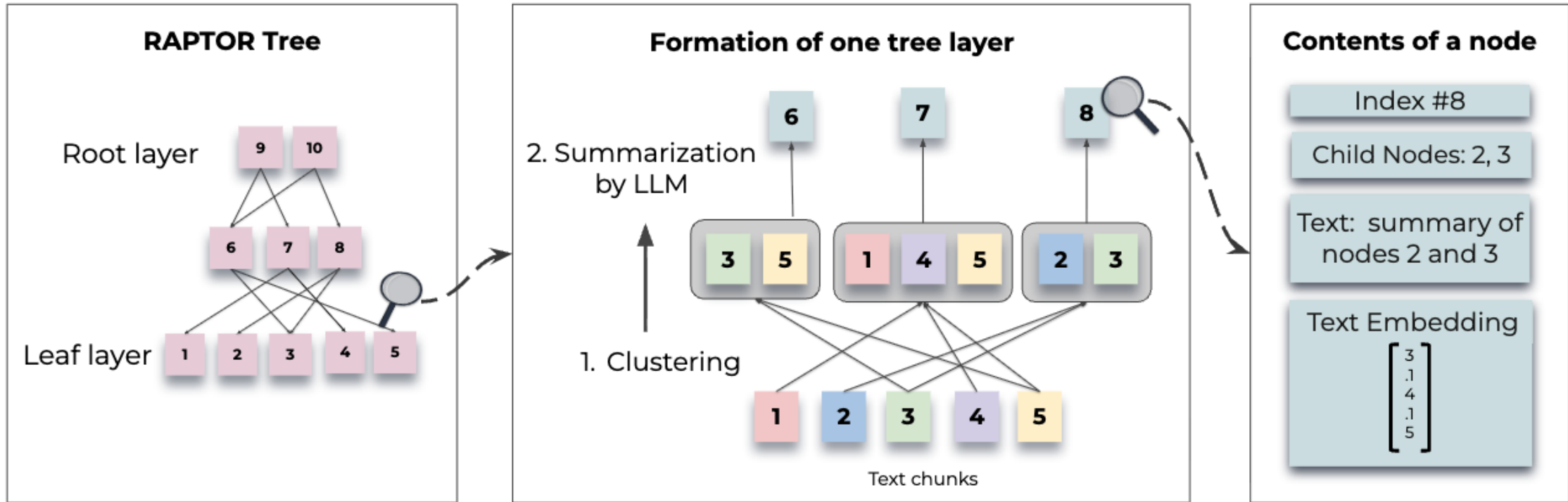
Retrieval Results with Long Late Chunking for Different Chunk Sizes

	SciFact			NFCorpus			FiQA			TRECCOVID			AVG
	J2s	J3	Nom	J2s	J3	Nom	J2s	J3	Nom	J2s	J3	Nom	
Fixed-Size Boundaries (256 Tokens per Chunk)													
Naive	64.2	71.8	70.7	23.5	35.6	35.3	33.3	46.3	37.0	63.4	73.0	72.9	52.2
Late	66.1	73.2	70.6	30.0	36.7	35.3	33.8	47.6	38.3	64.7	77.2	75.0	54.0
Sentence Boundaries (5 Sentences per Chunk)													
Naive	64.7	71.4	71.3	28.3	35.8	34.7	30.4	43.7	35.1	66.5	72.4	74.2	52.4
Late	65.2	73.2	71.4	30.0	36.6	35.5	33.9	48.0	37.7	66.6	76.5	76.8	54.3



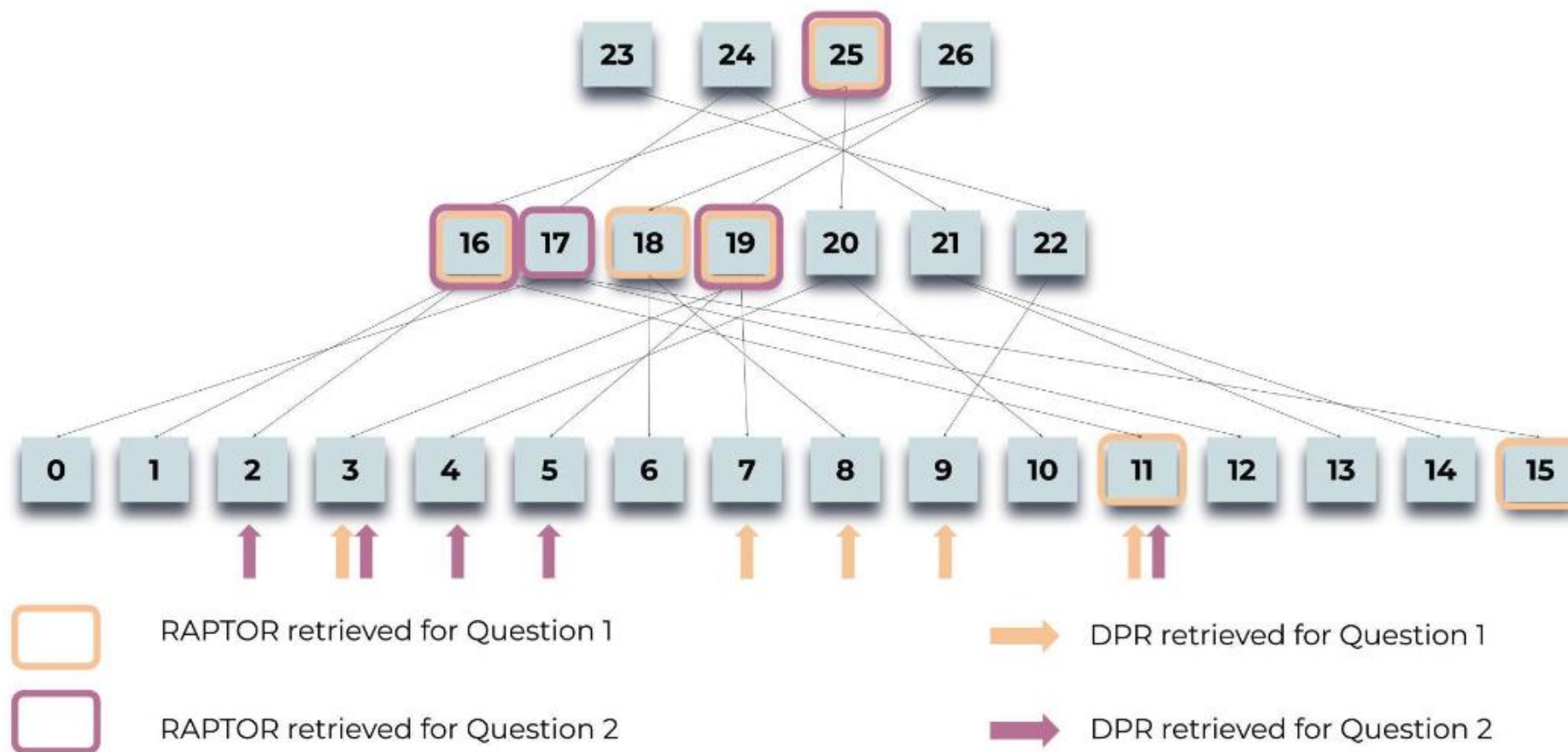
Hierarchical/Tree-Structured Indexing: RAPTOR

(Sarathi et al., 2025)



- For questions whose answer is spread across a document, single chunk representation can't help
- **Build a tree, bottom up:** embed chunks → cluster them → summarize each cluster with an LLM → embed the summaries → repeat
- **Retrieve across levels:** a detail question matches a leaf, a thematic question matches a node

Hierarchical/Tree-Structured Indexing: RAPTOR

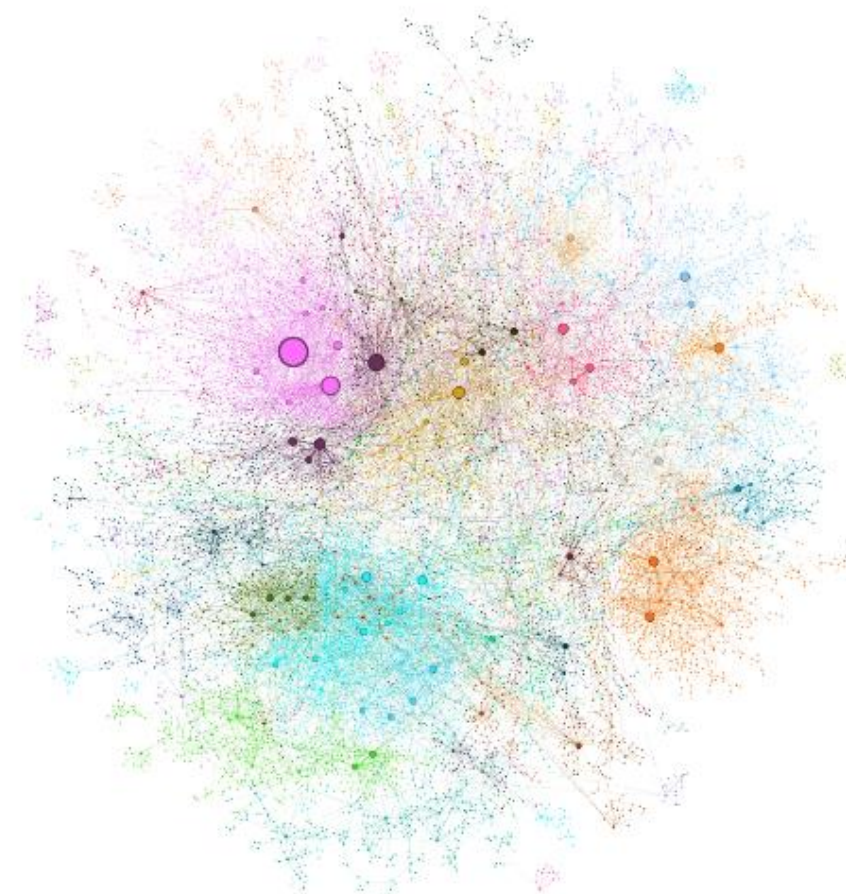
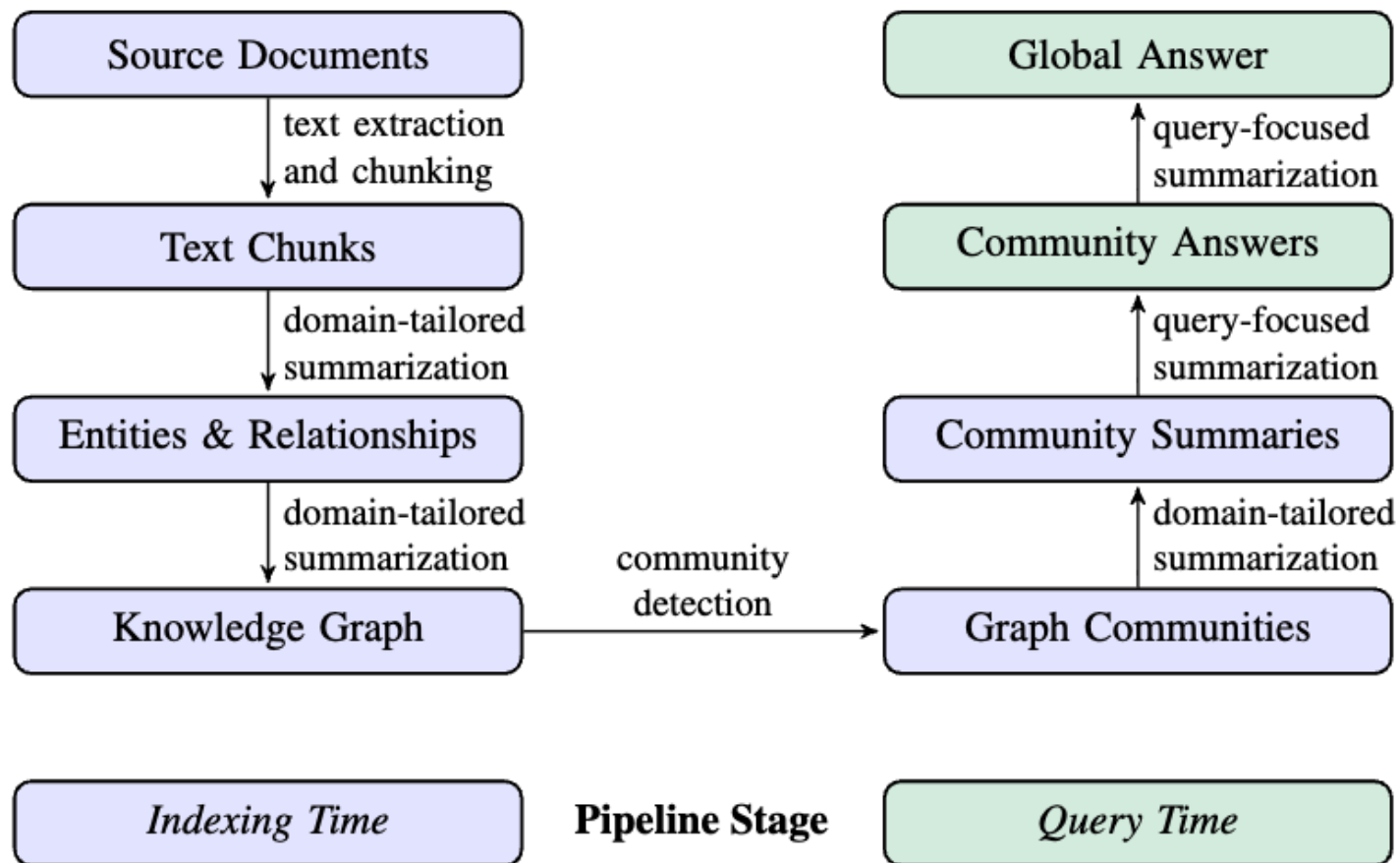


Cinderella queries: Q1 “*central theme of the story?*” and Q2 “*how did she find a happy ending?*”

Highlighted = RAPTOR’s selection; arrows = DPR’s leaves.

RAPTOR’s context usually already contains DPR’s, directly or inside a summary.

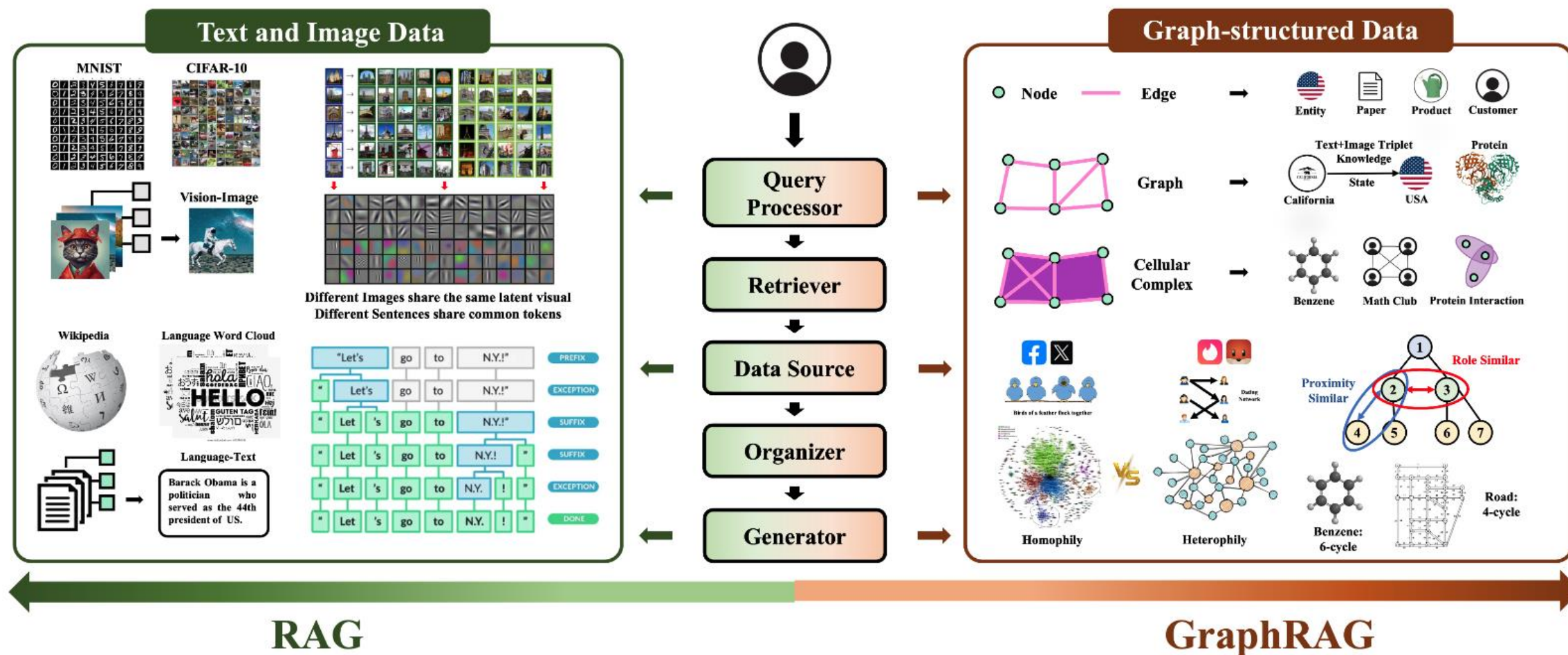
GraphRAG: query-focused over LLM-built graph



Graph communities detected

(Edge et al., 2024)

GraphRAG: More Variants (Han et al., 2025)

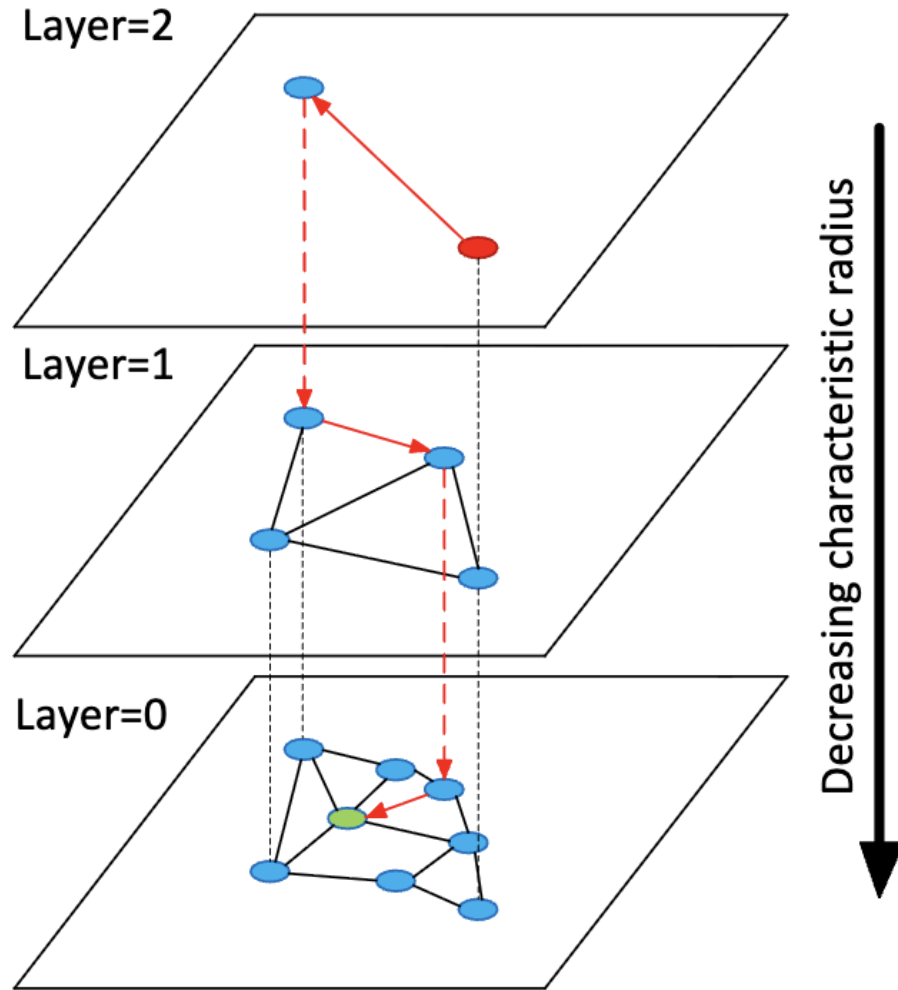


Vector Search

- **The task:** given a query vector, return the top-k of N chunk vectors
- **Exact search:** brute-force scan, $O(N \cdot d)$ per query
- **Approximate nearest neighbor (ANN):** trade a little recall for a lot of speed
 - **Inverted File Index (IVF):** cluster the index, probe only the nearest centroids
 - **Product Quantization (PQ):** compress each vector into a short code
 - **HNSW:** multi-layer proximity graph, greedy descent from a coarse layer

HNSW

(Hierarchical Navigable Small World)

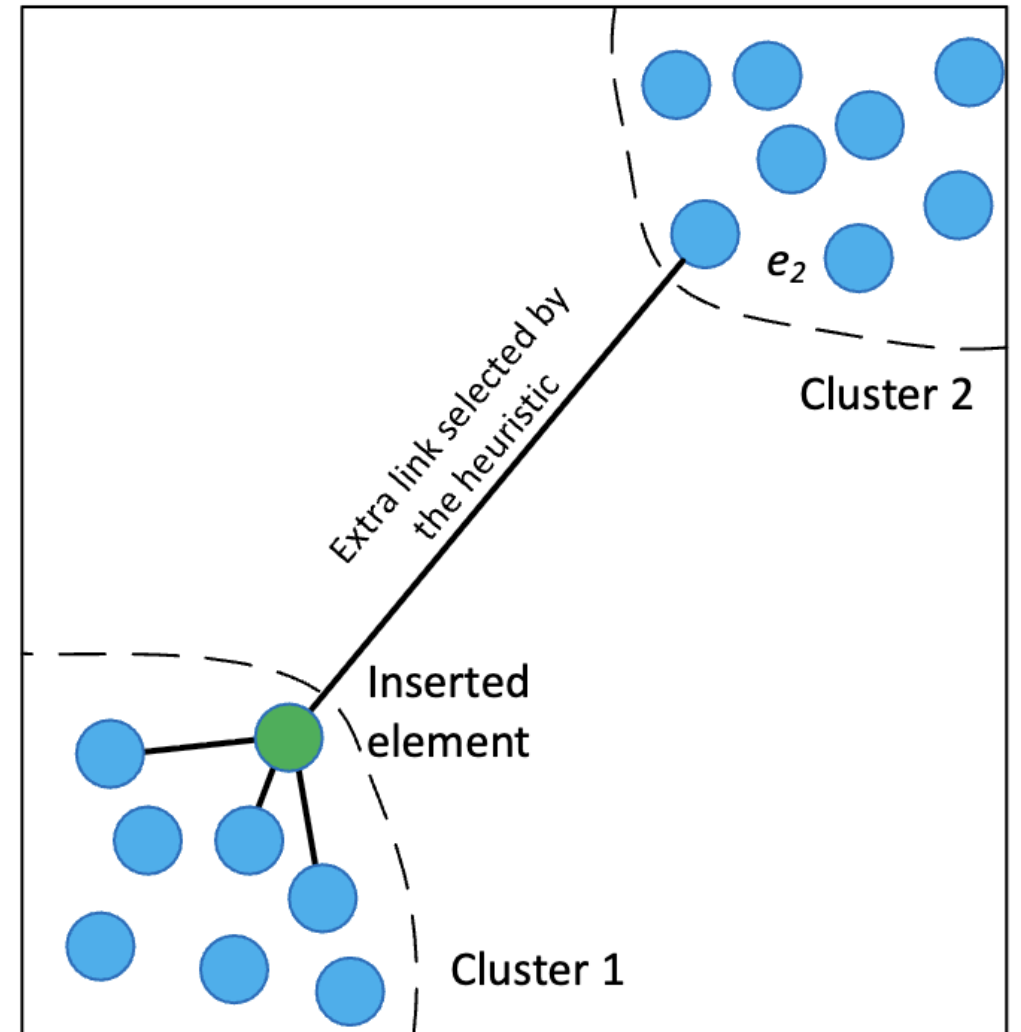


- **Layers:** all vectors at the bottom, thinner samples above
- **Search:** start at the top, greedy hop toward the query, drop a layer when stuck
- **Fast:** $\sim \log N$ hops, high recall

HNSW

How the graph is built (backup)

- **Insert** one vector at a time
- **Pick its top layer at random:** most nodes stay in layer 0, few reach the top
- **Link it at each layer:** M nearest neighbors, plus a heuristic link across to another cluster so the greedy walk cannot get trapped



Lecture Overview

- Why RAG (5 mins)
- The RAG pipeline (10 mins)
- Indexing: chunking, contextual retrieval (15 mins)
- Retrieval: BM25, DPR, ColBERT (20 mins)
- Agentic RAG: Self-RAG, Search-R1 (10 mins)
- Failure modes (15 mins)

Different Types of Retrievers

How should we retrieve relevant passages for our retrieval system?

- Word-overlap (BM25)
- Vector retrieval (DPR, Sentence vectors)
- Other, neural systems (ColBERT)

BM25: the most widely used (even today)

$$\text{score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{\text{How often } q_i \text{ occurs in } D \cdot (k_1 + 1)}{\text{How rare } q_i \text{ is} \cdot \left(f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}} \right) \right)}$$

$$\text{IDF}(q_i) = \ln \left(\frac{\text{Docs in the collection} \cdot N - n(q_i) + 0.5}{\text{Docs that contain } q_i \cdot n(q_i) + 0.5} \right)$$

Doc length vs. the average

IDF Comparison and Term Frequency Saturation

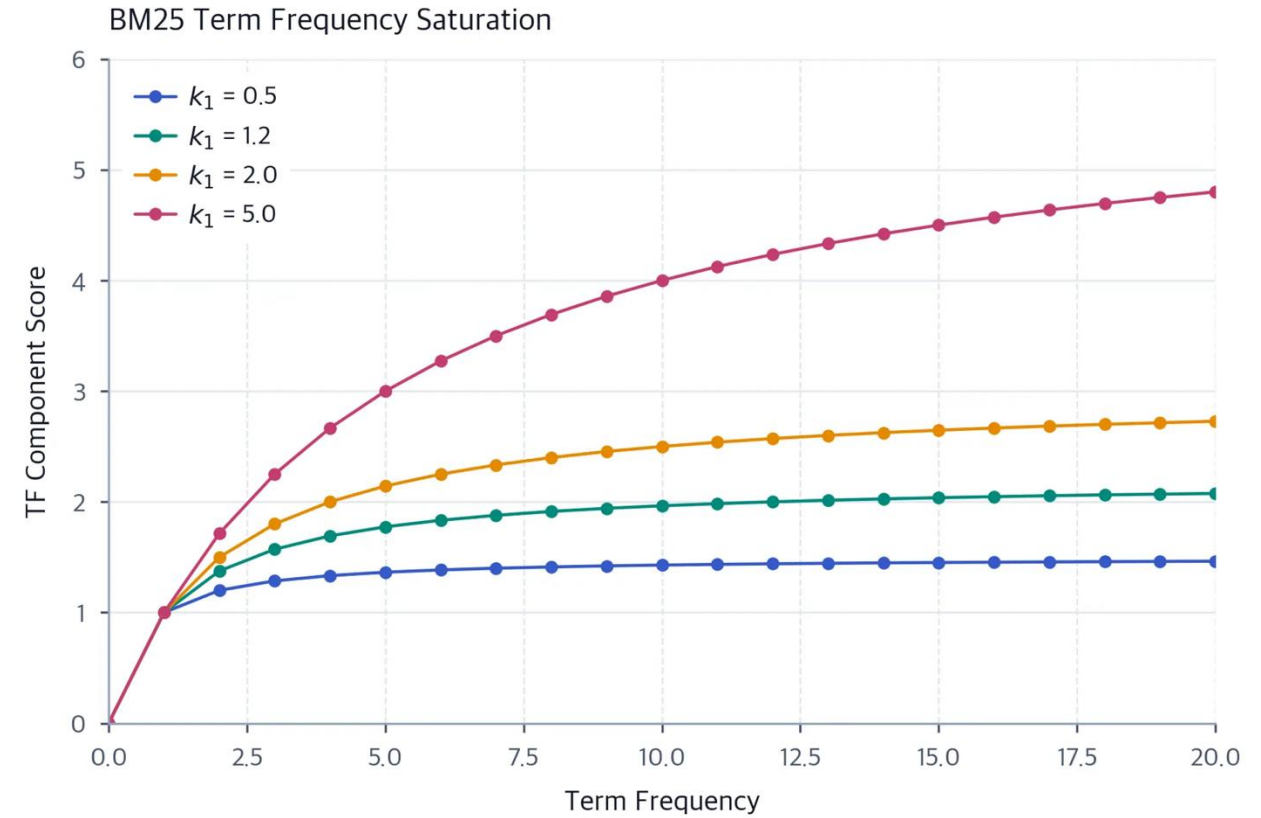
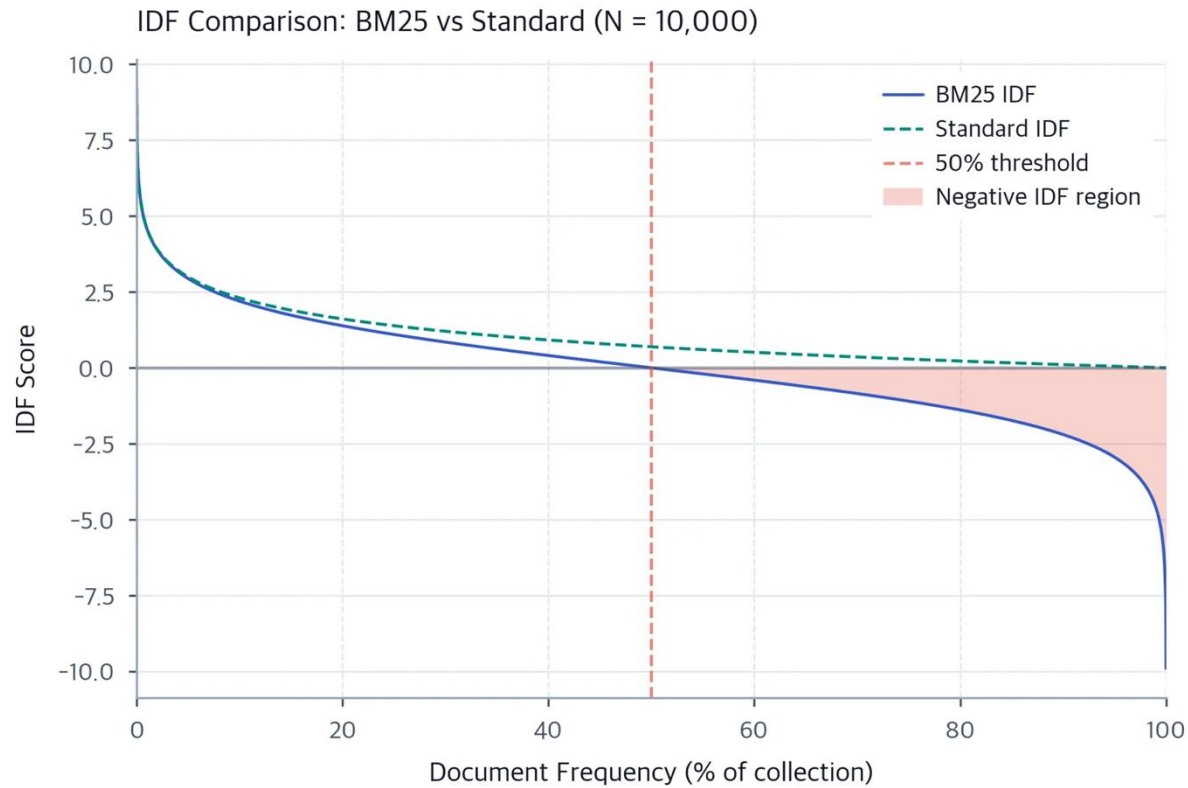
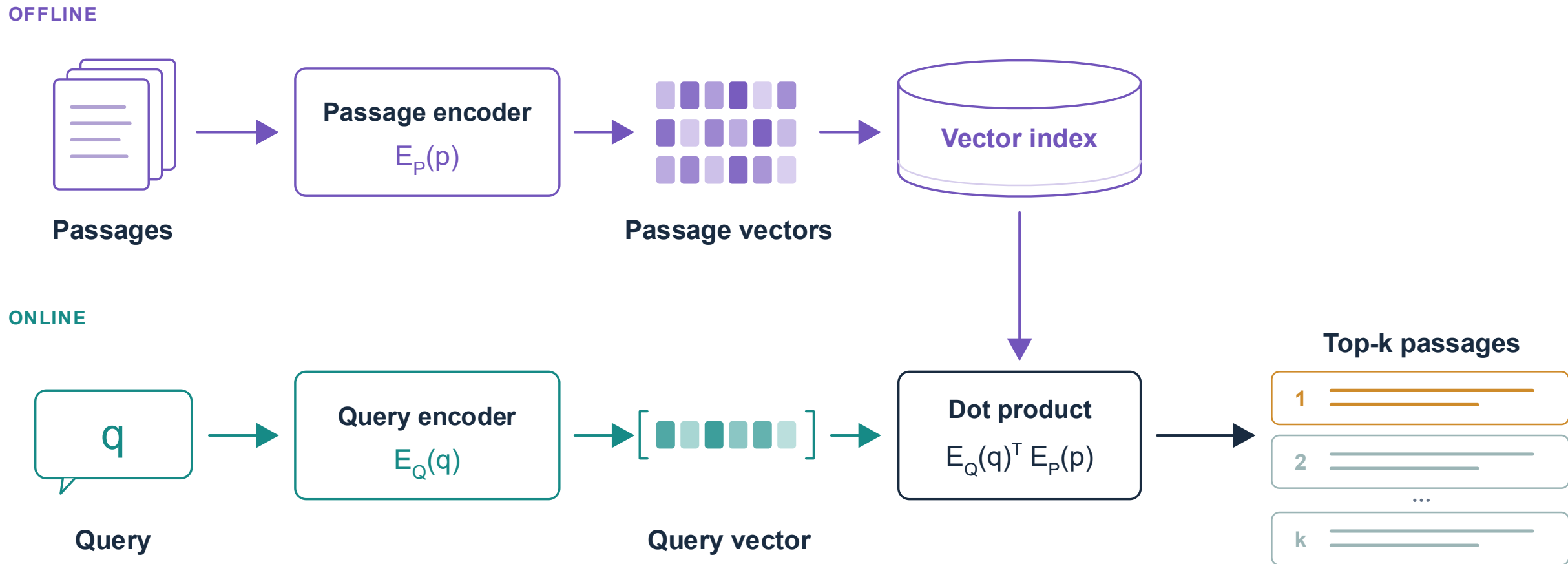
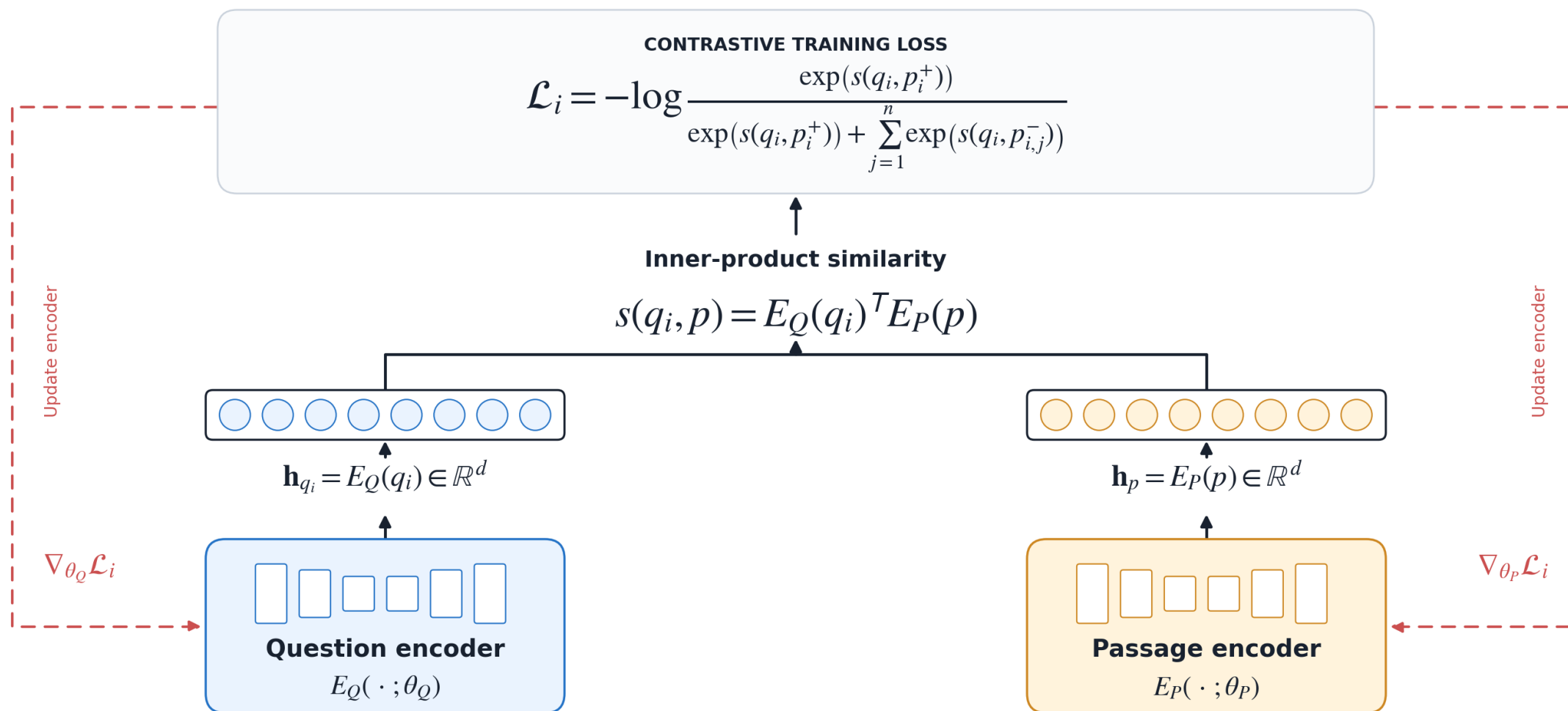


Figure credit to <https://mbrenndoerfer.com/writing/bm25-search-algorithm-elasticsearch-implementation>

DPR: dense passage retrieval (Karpukhin et al., 2020)



DPR: dense passage retrieval (Karpukhin et al., 2020)



Scoring a Retriever: Hit@k, Failed@k, nDCG@k

$$\text{Hit}@k = \frac{1}{|Q|} \sum_{q \in Q} \mathbb{1}[R_q \cap \text{Top}_k(q) \neq \emptyset]$$

1 if any relevant passage is in the top k, else 0

Position in the top k does not matter

$$\text{Failed}@k = \frac{1}{|Q|} \sum_{q \in Q} \mathbb{1}[R_q \cap \text{Top}_k(q) = \emptyset] = 1 - \text{Hit}@k$$

Queries with no relevant passage in the top k

$$\text{nDCG}@k = \frac{1}{|Q|} \sum_{q \in Q} \frac{\text{DCG}_q@k}{\text{IDCG}_q@k}$$

Normalized by the ideal ranking's DCG

$$\text{DCG}_q@k = \sum_{i=1}^k \frac{\text{rel}_{q,i}}{\log_2(i+1)}$$

Relevance of the passage at rank i

Discounted by rank position

Scoring a Retriever: MRR@k, Recall@k

Average over all queries

Rank of the first relevant passage

$$\text{MRR@}k = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q}$$

Counts as 0 if not in the top k

Relevant passages found in the top k

$$\text{Recall@}k = \frac{1}{|Q|} \sum_{q \in Q} \frac{|R_q \cap \text{Top}_k(q)|}{|R_q|}$$

All relevant passages for the query

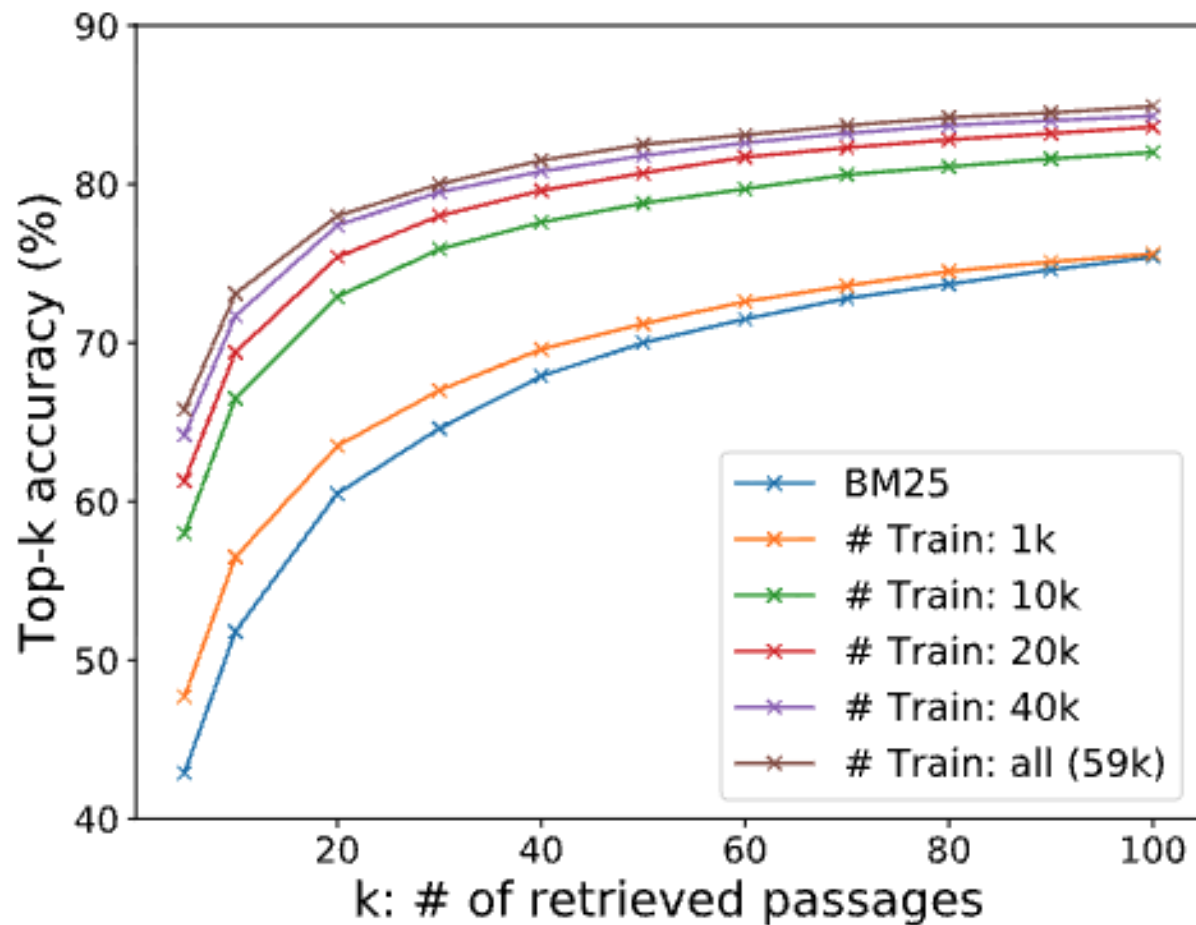
DPR beats BM25 on 4 of 5 datasets (Karpukhin et al., 2020)

Training: contrastive loss with in-batch negatives + BM25 hard negatives

Training	Retriever	Top-20					Top-100				
		NQ	TriviaQA	WQ	TREC	SQuAD	NQ	TriviaQA	WQ	TREC	SQuAD
None	BM25	59.1	66.9	55.0	70.9	68.8	73.7	76.7	71.1	84.1	80.0
Single	DPR	78.4	79.4	73.2	79.8	63.2	85.4	85.0	81.4	89.1	77.2
	BM25 + DPR	76.6	79.8	71.0	85.2	71.5	83.8	84.5	80.5	92.7	81.3
Multi	DPR	79.4	78.8	75.0	89.1	51.6	86.0	84.7	82.9	93.9	67.6
	BM25 + DPR	78.0	79.9	74.7	88.5	66.2	83.9	84.4	82.3	94.1	78.6

Top-20 & Top-100 retrieval accuracy on test sets. Single and Multi denote that our Dense Passage Retriever (DPR) was trained using individual or combined training datasets

1,000 QA pairs are enough to beat BM25



No extra pretraining: DPR also beats ORQA and REALM end to end

Training	Model	NQ	TriviaQA	WQ	TREC
Single	BM25+BERT (Lee et al., 2019)	26.5	47.1	17.7	21.3
Single	ORQA (Lee et al., 2019)	33.3	45.0	36.4	30.1
Single	HardEM (Min et al., 2019a)	28.1	50.9	-	-
Single	GraphRetriever (Min et al., 2019b)	34.5	56.0	36.4	-
Single	PathRetriever (Asai et al., 2020)	32.6	-	-	-
Single	REALM _{Wiki} (Guu et al., 2020)	39.2	-	40.2	46.8
Single	REALM _{News} (Guu et al., 2020)	40.4	-	40.7	42.9
Single	BM25	32.6	52.4	29.9	24.9
	DPR	41.5	56.8	34.6	25.9
	BM25+DPR	39.0	57.0	35.2	28.0
Multi	DPR	41.5	56.8	42.4	49.4
	BM25+DPR	38.8	57.9	41.1	50.6

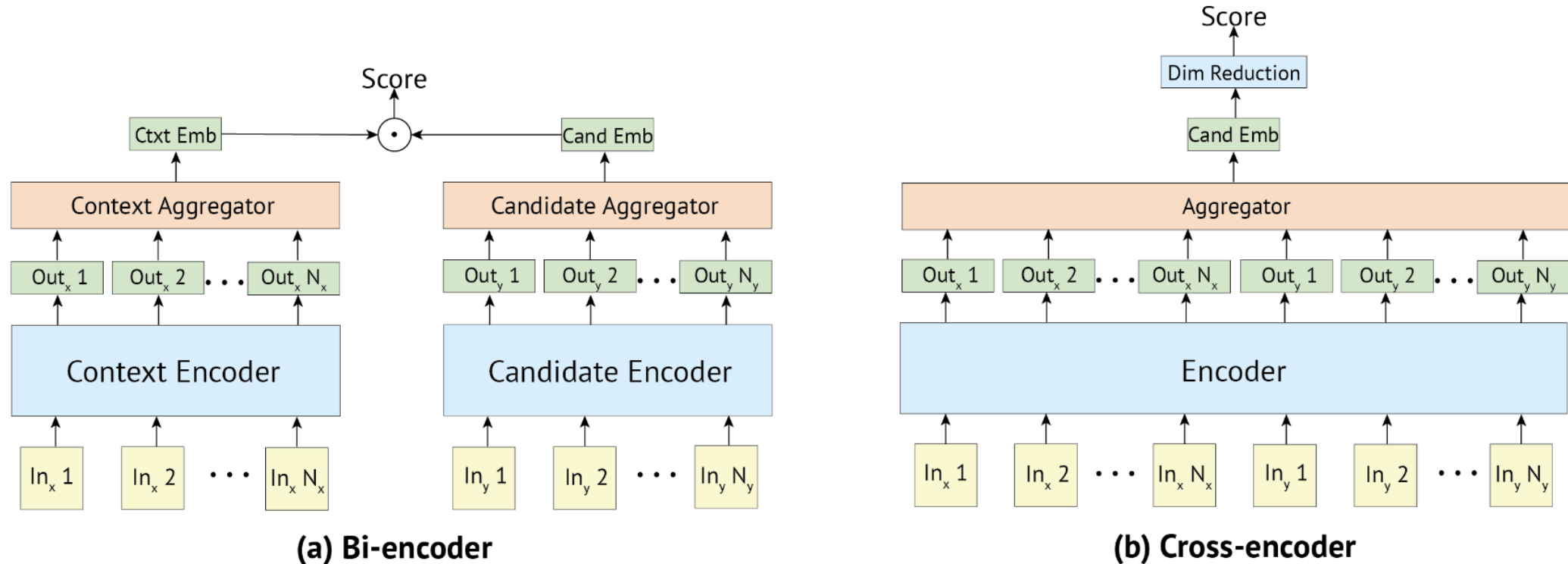
Hybrid Search and Reciprocal Rank Fusion

- The two approaches fail differently: BM25 misses paraphrase, dense retrieval misses rare literals (e.g., error codes, product IDs)
- **Hybrid search:** run both retrievers, then fuse the two ranked lists
- **Reciprocal rank fusion:** merge positions instead of raw scores
 - no per-corpus tuning and works with any number of retrievers

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)} \quad k = 60$$

Cross-Encoders and Reranking (Humeau et al., 2020)

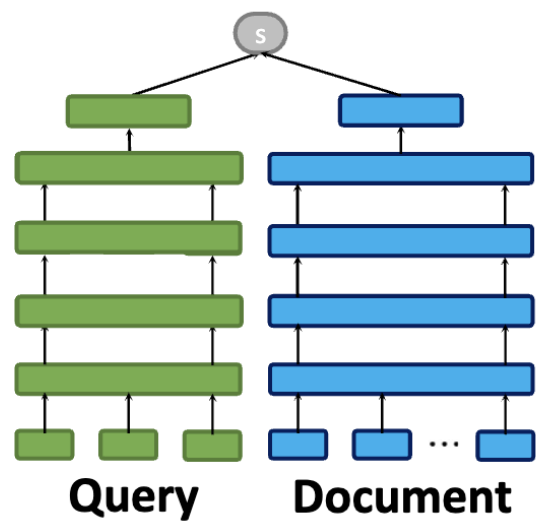
- **Bi-encoder (retrieval):** query and passage encoded separately
- **Cross-encoder (reranking):** query and passage go through the model together, full attention across both



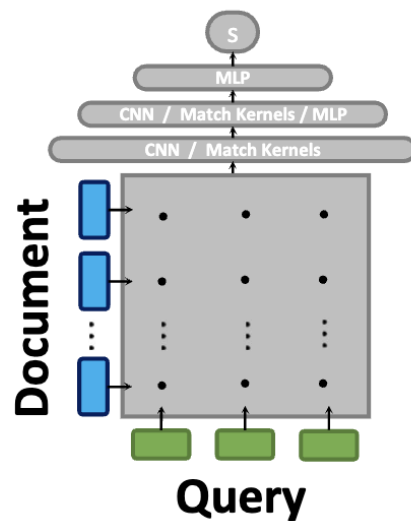
Late Interaction: Score Tokens, Not Documents

Late interaction: keep one vector per token, encode query and passage independently, and interact only at scoring time; scoring is MaxSim

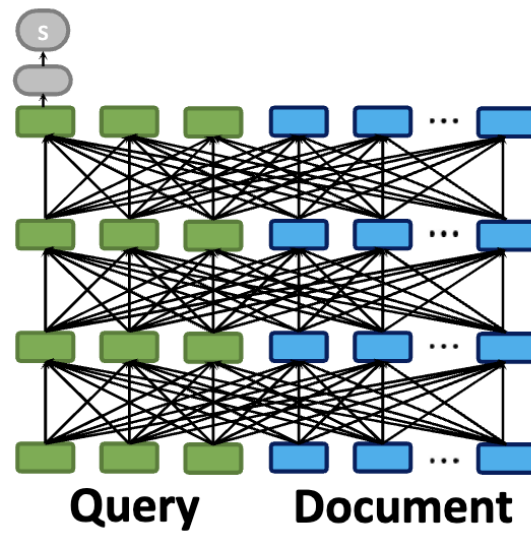
$$S_{q,d} = \sum_{i \in q} \max_j \mathbf{q}_i \cdot \mathbf{d}_j$$



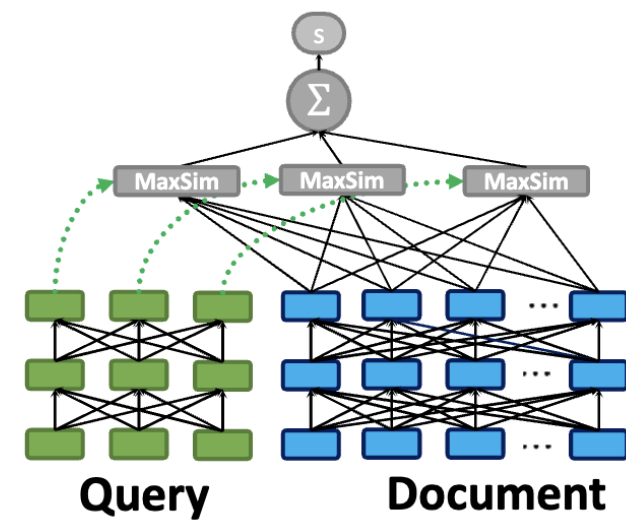
(a) Representation-based Similarity
(e.g., DSSM, SNRM)



(b) Query-Document Interaction
(e.g., DRMM, KNRM, Conv-KNRM)

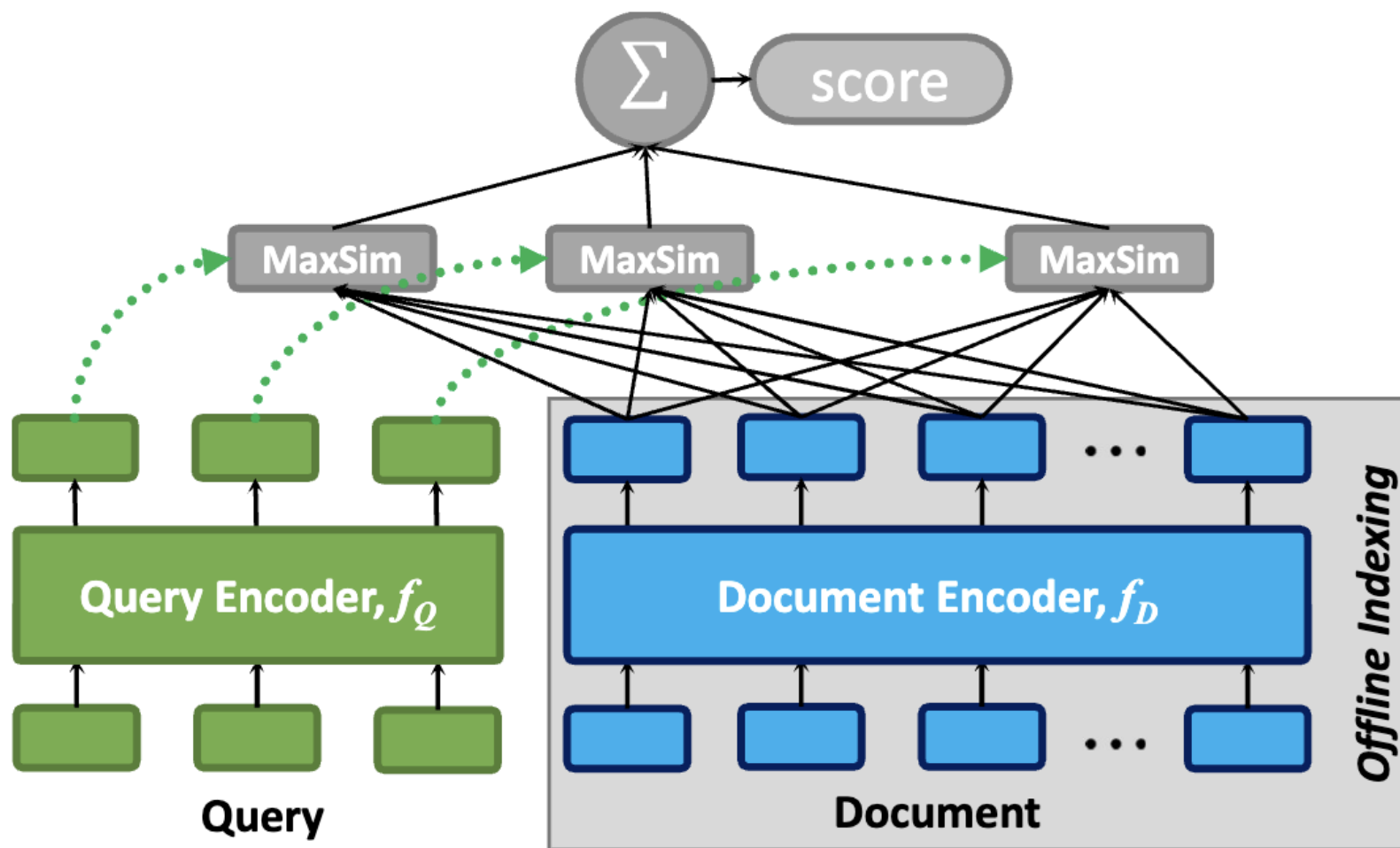


(c) All-to-all Interaction
(e.g., BERT)



(d) Late Interaction
(i.e., the proposed ColBERT)

General Architecture of ColBERT



ColBERT vs. Other Approaches

Method	MRR@10 (Dev)	MRR@10 (Local Eval)	Latency (ms)	Recall@50	Recall@200	Recall@1000
BM25 (official)	16.7	-	-	-	-	81.4
BM25 (Anserini)	18.7	19.5	62	59.2	73.8	85.7
doc2query	21.5	22.8	85	64.4	77.9	89.1
DeepCT	24.3	-	62 (<i>est.</i>)	69 [2]	82 [2]	91 [2]
docTTTTTquery	27.7	28.4	87	75.6	86.9	94.7
ColBERT _{L2} (re-rank)	34.8	36.4	-	75.3	80.5	81.4
ColBERT _{L2} (end-to-end)	36.0	36.7	458	82.9	92.3	96.8

Method	MRR@10 (Dev)	MRR@10 (Eval)	Re-ranking Latency (ms)	FLOPs/query
BM25 (official)	16.7	16.5	-	-
KNRM	19.8	19.8	3	592M (0.085×)
Duet	24.3	24.5	22	159B (23×)
fastText+ConvKNRM	29.0	27.7	28	78B (11×)
BERT _{base} [25]	34.7	-	10,700	97T (13,900×)
BERT _{base} (our training)	36.0	-	10,700	97T (13,900×)
BERT _{large} [25]	36.5	35.9	32,900	340T (48,600×)
ColBERT (over BERT _{base})	34.9	34.9	61	7B (1×)

Fast vs Slow Retrievers

Retriever	Precomputed offline	Query latency	Wins on	Breaks on
BM25 (sparse)	Inverted index	62 ms fast	Exact terms: IDs, names, error codes	Paraphrase, synonyms
DPR (bi-encoder)	One vector per passage	Tens of ms fast	Semantic match at corpus scale	Rare literals; needs labeled pairs
ColBERT (late interaction)	One vector per token	458 ms end-to-end middle	Token-level evidence at scale	Index size: tokens, not documents
Cross-encoder (BERT-base)	Nothing	10,700 ms per 1k passages slow	Accuracy on a shortlist	Cannot scan a corpus

Latencies from Khattab & Zaharia (2020), MS MARCO; DPR from Karpukhin et al. (2020), different hardware

Lecture Overview

- Why RAG (5 mins)
- The RAG pipeline (10 mins)
- Indexing: chunking, contextual retrieval (15 mins)
- Retrieval: BM25, DPR, ColBERT (20 mins)
- Agentic RAG: Self-RAG, Search-R1 (10 mins)
- Failure modes (15 mins)

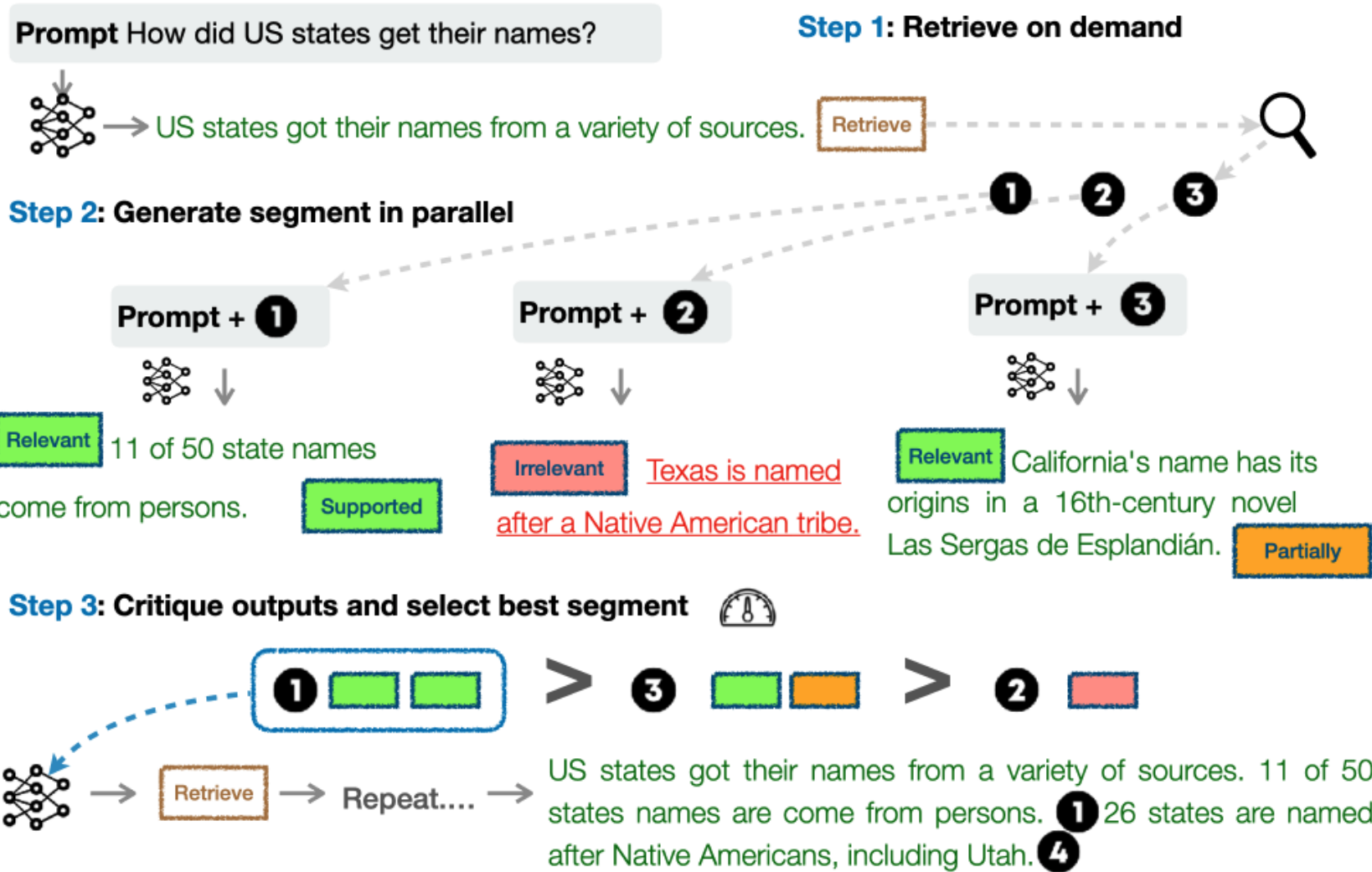
Agentic RAG: Retrieval inside the Loop

Everything so far answers one question well. None of it decides **whether** to retrieve, **what** to ask, or **when to stop**.



- **Retrieval should be an action the agent takes**
- **Three decisions:** (1) retrieve or answer directly; (2) what query to issue; (3) whether the result is good enough

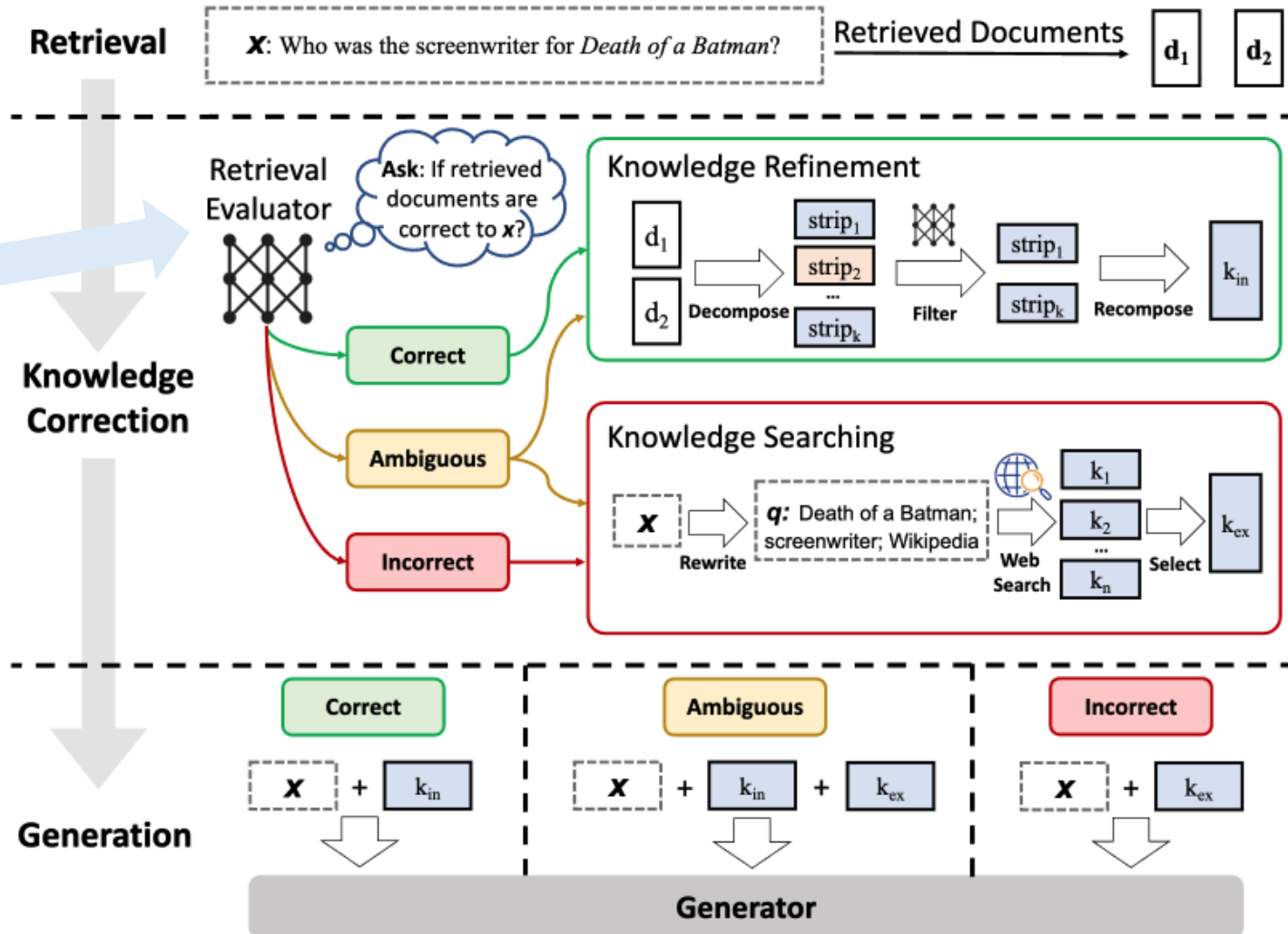
Self-RAG: Self-reflective RAG (Asai et al., 2024)



SELF-RAG learns to retrieve, critique, and generate text passages to enhance overall generation quality, factuality, and verifiability.

CRAG: a critic grades the retrieval (Yan et al., 2024)

To evaluate the relevance of the retrieved docs to the input, and estimate a confidence based on which different knowledge retrieval actions can be triggered.



Search-R1: learn the search policy with RL (Jin et al., 2025)

Question: Who is older, Jed Hoyer or John William Henry II?

Ground Truth: John William Henry II

<think> I need to determine if Jed Hoyer or John William Henry II is older. I'll search for their ages. </think>

<search> Jed Hoyer or John William Henry II </search>

<information> Doc 1(Title: "Jed Hoyer") career saves record with Sam Elias and Nick Miceli. During his time at Wesleyan University he was also initiated into and is a brother of Delta Kappa Epsilon. He spent Summer 1995 on the roster of the Waterbury Barons of the New England Collegiate Baseball League seeing innings on the mound and at shortstop. Hoyer worked in the admissions office and then the alumni/development office, and was also a baseball coach at the university after graduating. He worked in the admissions department of Kenyon College before joining the Red Sox at 28. He married Merrill Muckerman in June 2010 in ... </information>

<think> I found out that Jed Hoyer is a person. Now I need to find his age and compare it to John William Henry II's age. </think>

<search> John William Henry II </search>

<information> Doc 1(Title: "John W. Henry") the firm's management had fallen from \ \$2.5 billion in 2006 to less than \ \$100 million as of late 2012. As of July 2017, ""Forbes"" estimated his net worth to be \ \$2.6 billion. John William Henry II was born on September 13, 1949 in Quincy, Illinois. His parents were soybean farmers, and he split his time growing up between Illinois and Arkansas. His asthmatic condition at the age of 15 prompted his family to move to Apple Valley, California. After his graduation from Victor Valley High School in Victorville, he attended Victor Valley College, then the University of California (at Riverside, ... </information>

<think> I found out that John William Henry II was born on September 13, 1949. Now I need to find Jed Hoyer's birth year to compare. </think>

<search> Jed Hoyer birth year </search>

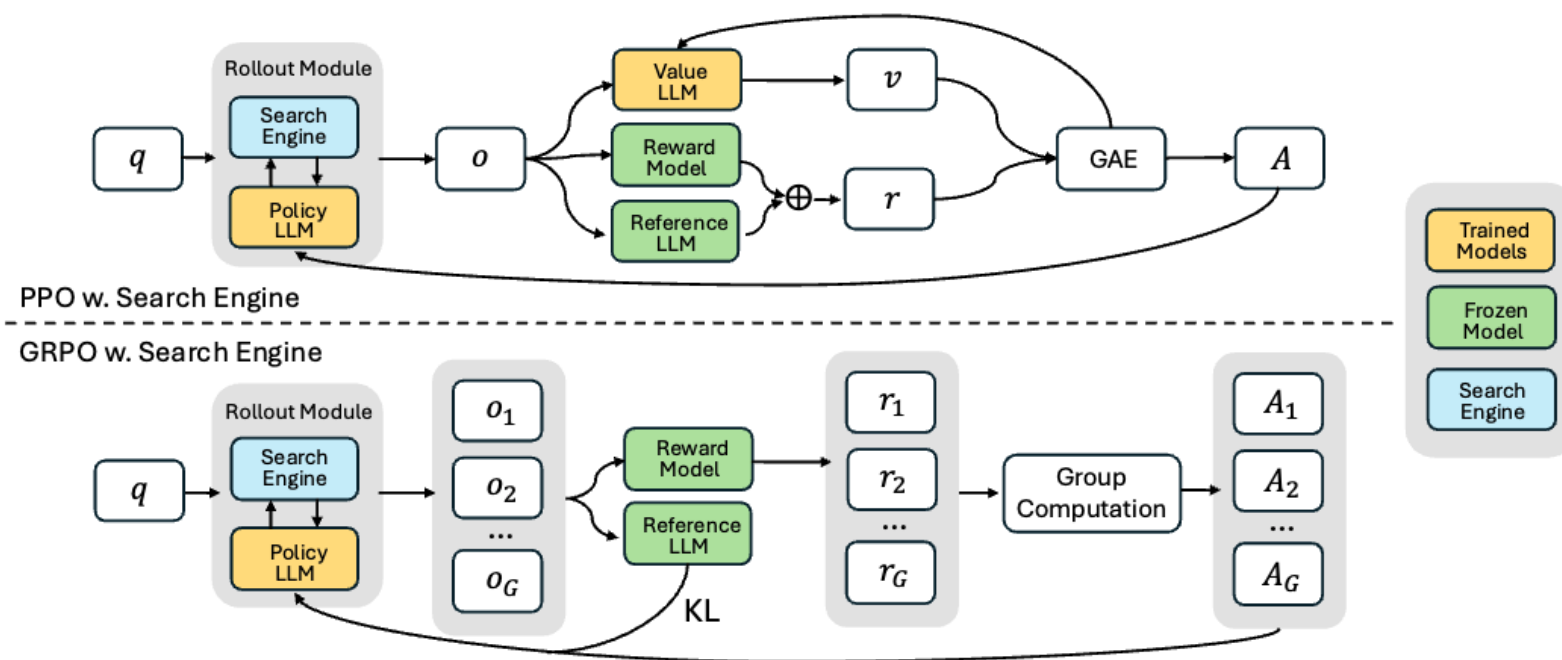
<information> Doc 1(Title: "Jed Hoyer") career saves record with Sam Elias and Nick Miceli. During his time at Wesleyan University he was also initiated into and is a brother of Delta Kappa Epsilon. He spent Summer 1995 on the roster of the Waterbury Barons of the New England Collegiate Baseball League seeing innings on the mound and at shortstop. Hoyer worked in the admissions office and then the alumni/development office, and was also a baseball coach at the university after graduating. He worked in the admissions department of Kenyon College before joining the Red Sox at 28. He married Merrill Muckerman in June 2010 in ... </information>

<think> I found out that Jed Hoyer was born on December 7, 1973. Now I can compare his birth year to John William Henry II's birth year. </think>

<answer> John William Henry II </answer>

Search-R1 optimizes LLM reasoning trajectories with multi-turn search interactions

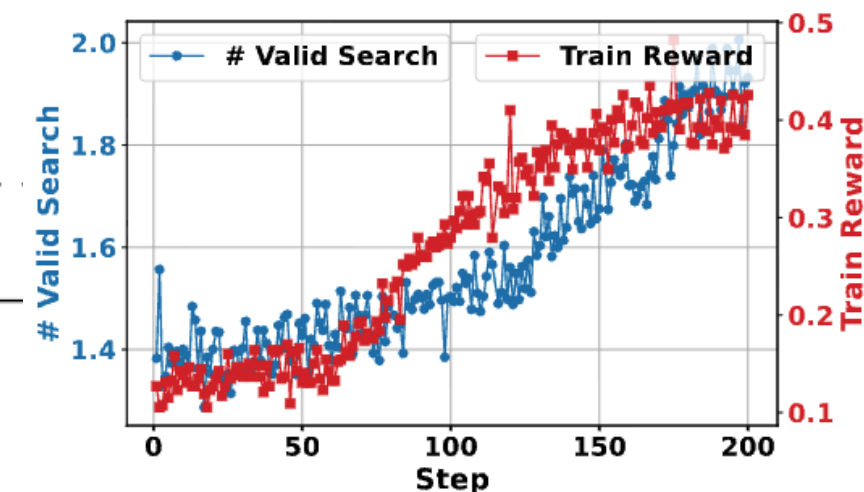
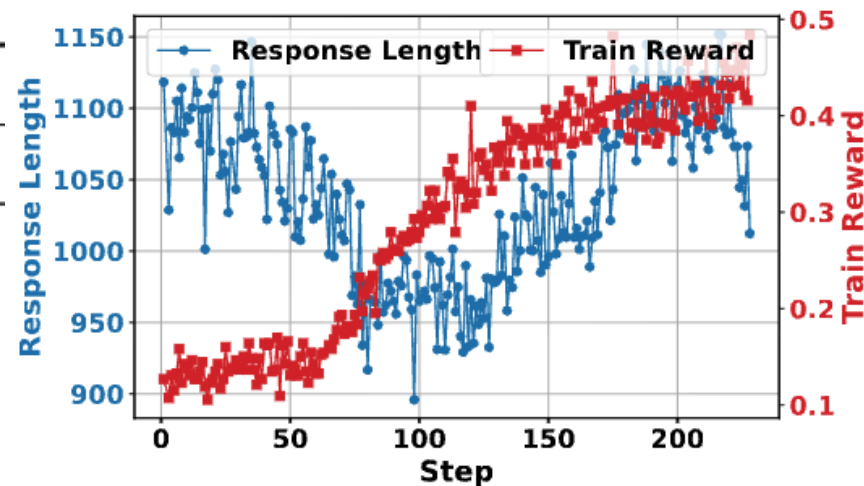
Search-R1: learn the search policy with RL (Jin et al., 2025)



Answer the given question. You must conduct reasoning inside `<think>` and `</think>` first every time you get new information. After reasoning, if you find you lack some knowledge, you can call a search engine by `<search>` query `</search>`, and it will return the top searched results between `<information>` and `</information>`. You can search as many times as you want. If you find no further external knowledge needed, you can directly provide the answer inside `<answer>` and `</answer>` without detailed illustrations. For example, `<answer>` xxx `</answer>`. Question: **question**.

Search-R1: learn the search policy with RL (Jin et al., 2025)

Methods	General QA				Multi-Hop QA	
	NQ [†]	TriviaQA [*]	PopQA [*]	HotpotQA [†]	2wiki [*]	Musique [*]
Qwen2.5-7b-Base/Instruct						
Direct Inference	0.134	0.408	0.140	0.183	0.250	0.031
CoT	0.048	0.185	0.054	0.092	0.111	0.022
IRCoT	0.224	0.478	0.301	0.133	0.149	0.072
Search-o1	0.151	0.443	0.131	0.187	0.176	0.058
RAG	0.349	0.585	0.392	0.299	0.235	0.058
SFT	0.318	0.354	0.121	0.217	0.259	0.066
R1-base	0.297	0.539	0.202	0.242	0.273	0.083
R1-instruct	0.270	0.537	0.199	0.237	0.292	0.072
Rejection Sampling	0.360	0.592	0.380	0.331	0.296	0.123
Search-R1-base	0.480	0.638	0.457	0.433	0.382	0.196
Search-R1-instruct	0.393	0.610	0.397	0.370	0.414	0.146



Lecture Overview

- Why RAG (5 mins)
- The RAG pipeline (10 mins)
- Indexing: chunking, contextual retrieval (15 mins)
- Retrieval: BM25, DPR, ColBERT (20 mins)
- Agentic RAG: Self-RAG, Search-R1 (10 mins)
- Failure modes (15 mins)

Issues with RAG

Retrieval-side



Recall ceiling

Missing evidence limits agent

Stale index

Embeddings not update

Permission leak

Use content not allowed to access

Reader-side



Lost in the middle

Position bias

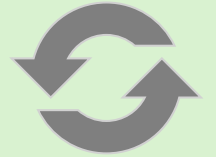
Distraction

Similar but irrelevant content

Evidence conflict

Sources or knowledge conflicts

Loop-side



Compounding error

Early mistakes affect later decision

No stopping rule

Lack clear conditions for finishing

Prompt injection

Retrieved text steers the agent